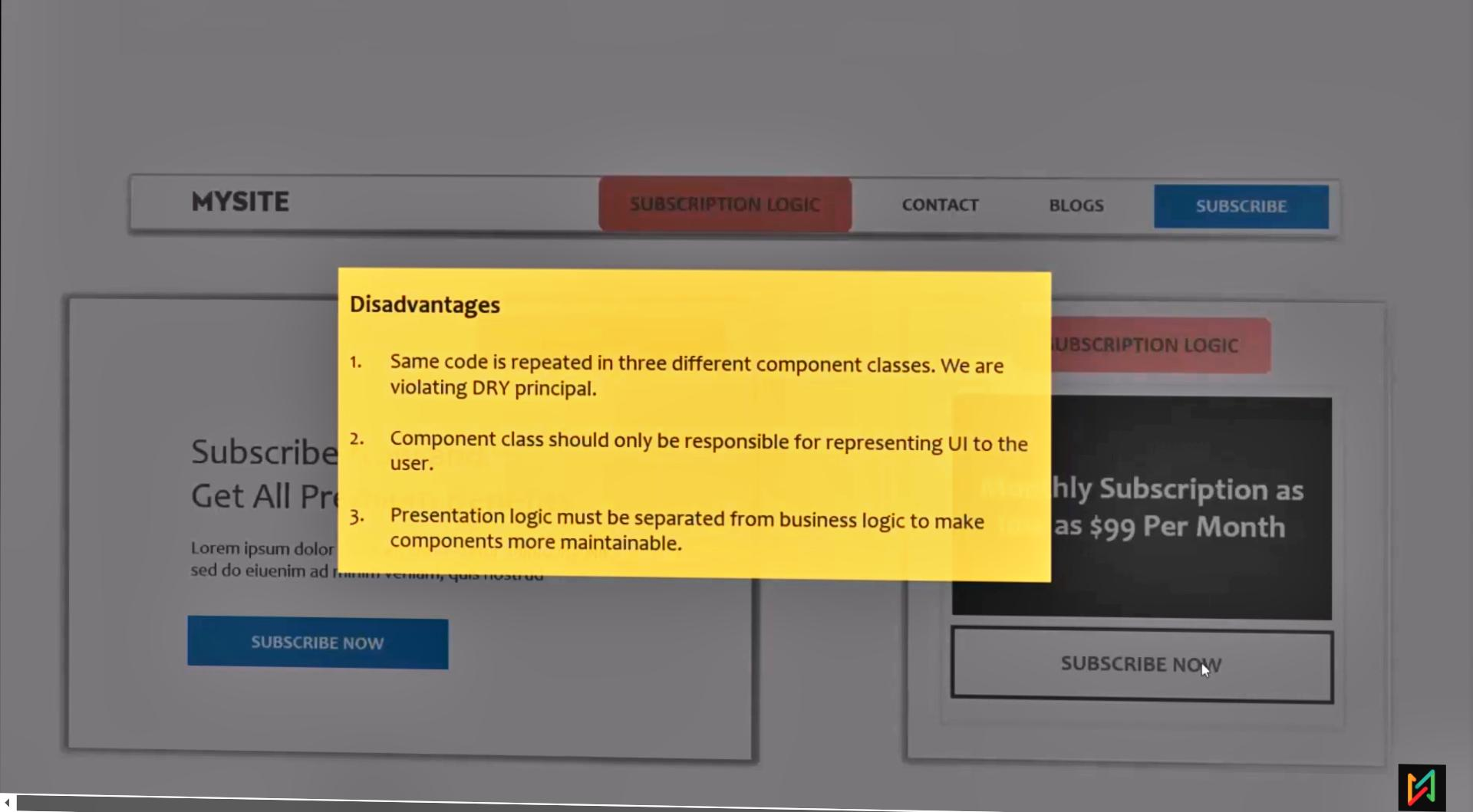
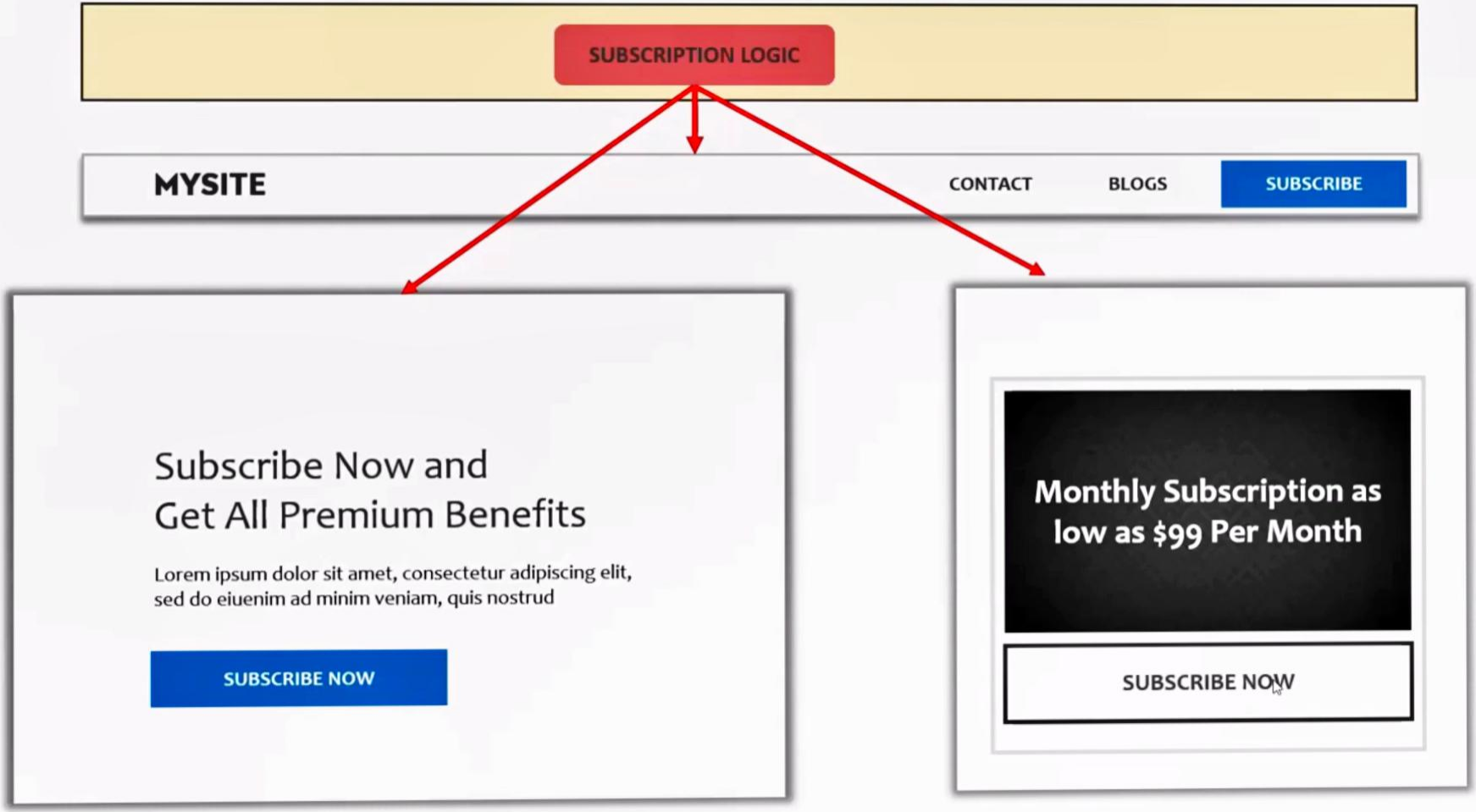
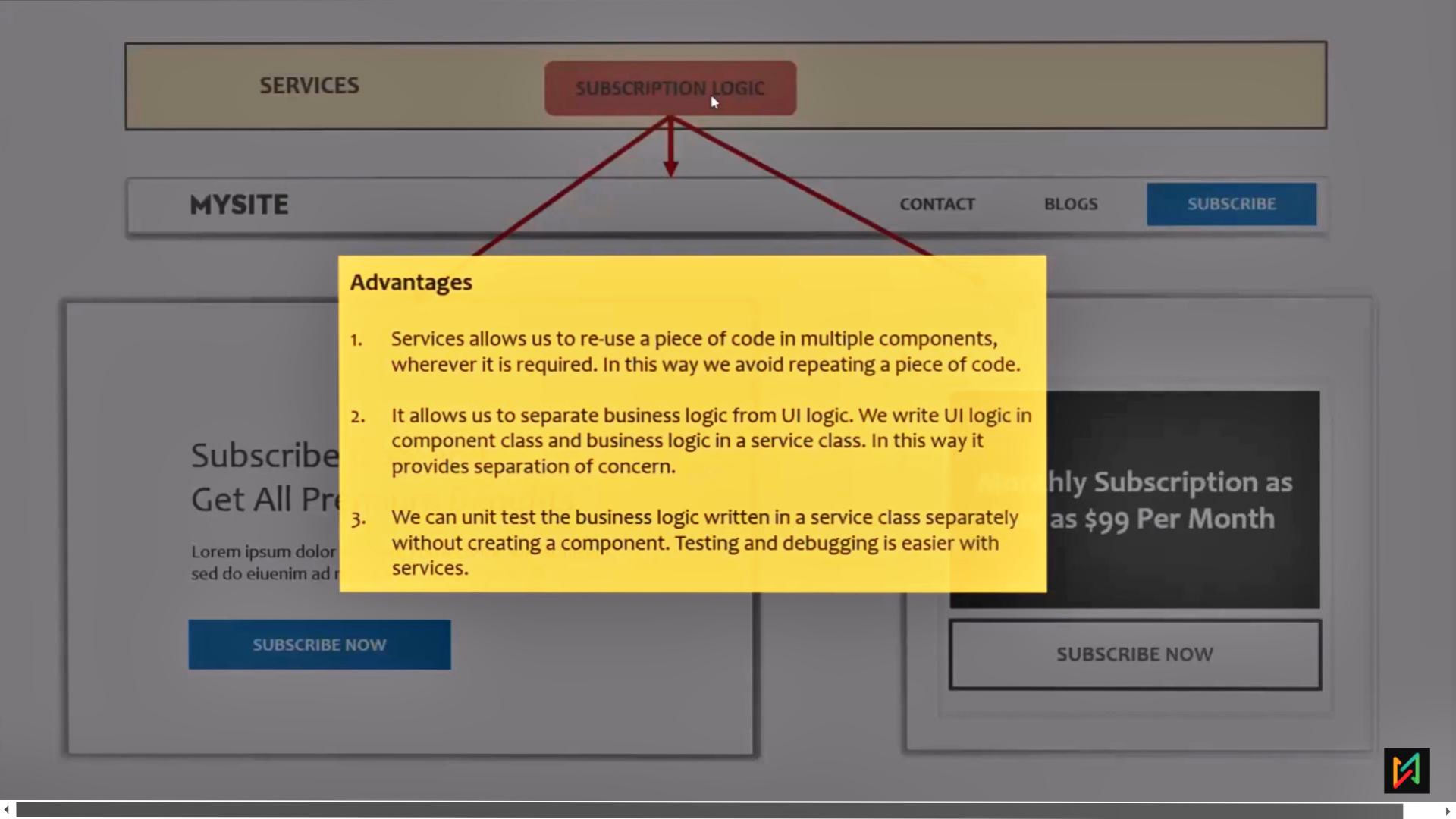
What is Service in Angular


MYSITE
SUBSCRIPTION LOGIC
CONTACT
BLOGS
SUBSCRIBE
Disadvantages
1. Same code is repeated in three different component classes. We are violating DRY principal.
2. Component class should only be responsible for representing UI to the user.
3. Presentation logic must be separated from business logic to make components more maintainable.
Subscribe
Get All Pr
Lorem ipsum dolor
sed do eiuenim ad
SUBSCRIBE NOW
UBSCRIPTION LOGIC
hly Subscription as
as $99 Per Month
SUBSCRIBE NOW


SUBSCRIPTION LOGIC
MYSITE
CONTACT
BLOGS
SUBSCRIBE
Subscribe Now and
Get All Premium Benefits
Lorem ipsum dolor sit amet, consectetur adipiscing elit,
sed do eiuenim ad minim veniam, quis nostrud
SUBSCRIBE NOW
Monthly Subscription as
low as $99 Per Month
SUBSCRIBE NOW


SERVICES
SUBSCRIPTION LOGIC
MYSITE
CONTACT
BLOGS
SUBSCRIBE
Advantages
1. Services allows us to re-use a piece of code in multiple components, wherever it is required. In this way we avoid repeating a piece of code.
2. It allows us to separate business logic from UI logic. We write UI logic in component class and business logic in a service class. In this way it provides separation of concern.
3. We can unit test the business logic written in a service class separately without creating a component. Testing and debugging is easier with services.
Subscribe
Get All Pr
Lorem ipsum dolor
sed do eiuenim ad
SUBSCRIBE NOW
hly Subscription as
as $99 Per Month
SUBSCRIBE NOW

A service in Angular is a re-usable TypeScript class that can be used in multiple components across our Angular application.

We can also use services to communicate between two non-related components in Angular

Dependency Injection (DI)

A dependency is a relationship between two software components where one component relies on the other component to work properly.

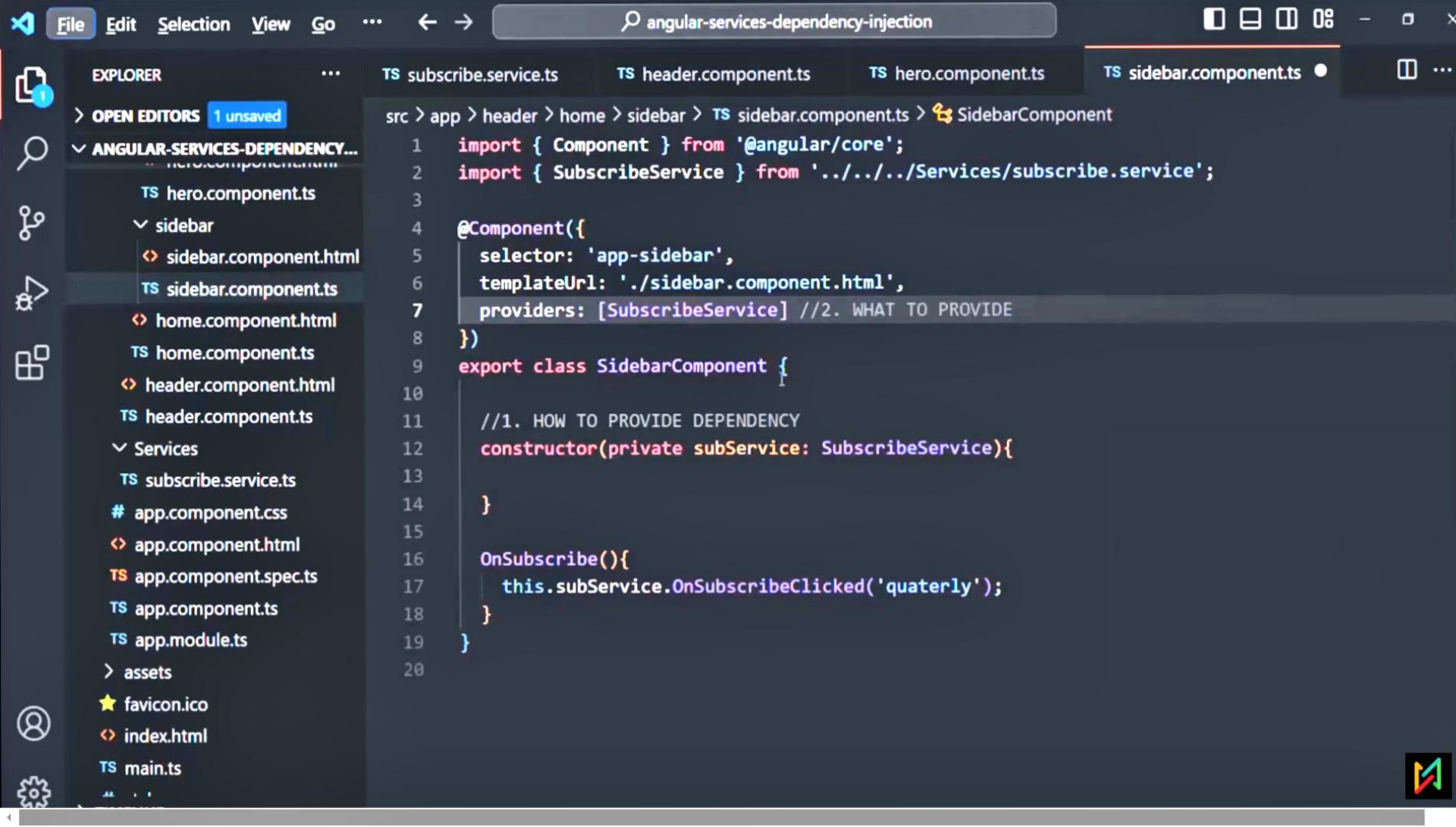
Disadvantage of not using Dependency Injection (DI)

1. Without dependency injection, a class is tightly coupled with its dependency. This makes a class non-flexible. Any change in dependency forces us to change the class implementation.

2. It makes testing of class difficult. Because if the dependency changes, the class has to change. And when the class changes, the unit test mock code also has to change.

```
@Component({
    selector: 'app-header',
    template: 'header.component.html',
    provider: [SubscribeService]
})
export class HeaderComponent{
    subService: SubscribeService;

    constructor(subService: SubscribeService){
        this.subservice = subservice;
    }

    OnSubscribe(){
        this.SubService.OnSubscribeClicked();
    }
}
```

Injector

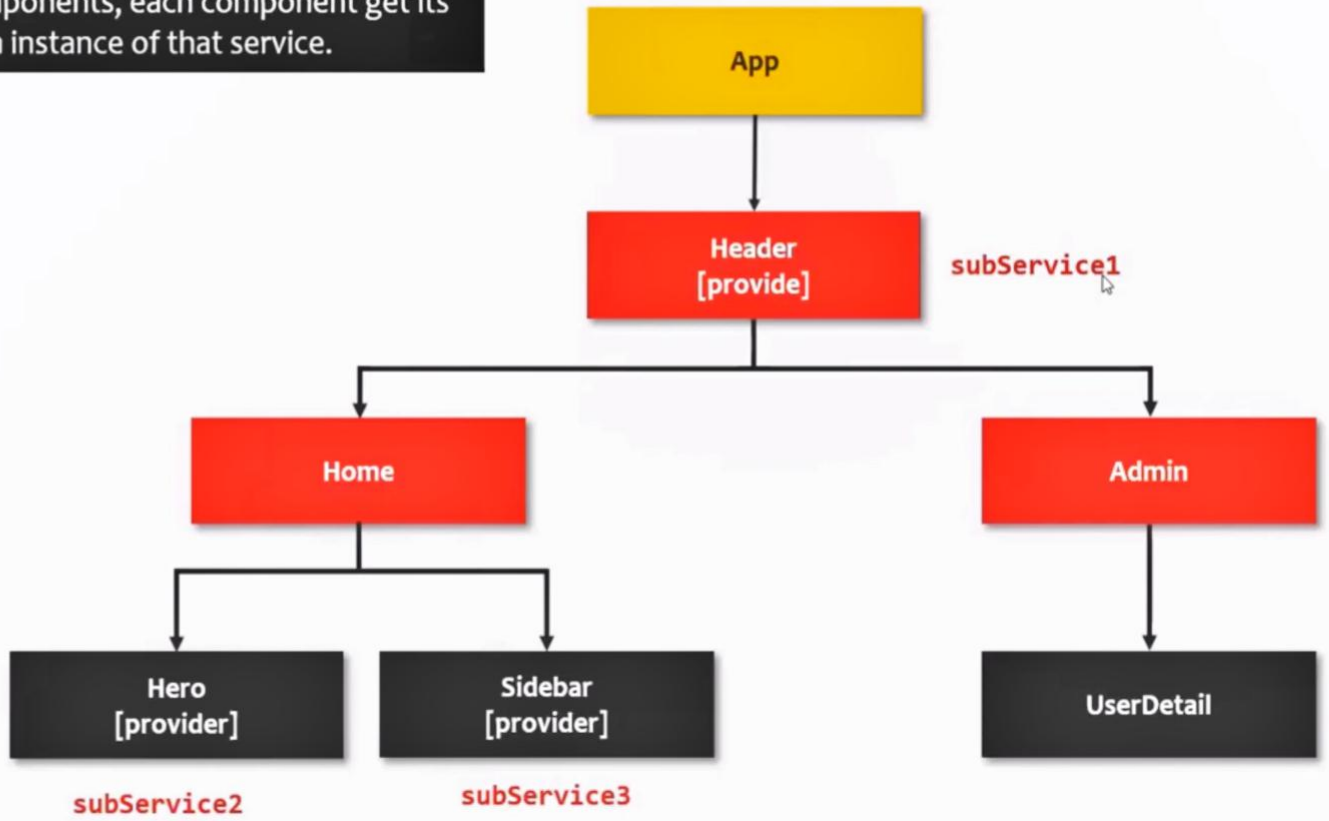

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
OPEN EDITORS 1 unsaved
ANGULAR-SERVICES-DEPENDENCY...
hero.component.ts
sidebar
sidebar.component.html
sidebar.component.ts
home.component.html
home.component.ts
header.component.html
header.component.ts
Services
subscribe.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
favicon.ico
index.html
main.ts
subscribe.service.ts
header.component.ts
hero.component.ts
sidebar.component.ts
src > app > header > home > sidebar > sidebar.component.ts > SidebarComponent
1 import { Component } from '@angular/core';
2 import { SubscribeService } from '../../../Services/subscribe.service';
3
4 @Component({
5 selector: 'app-sidebar',
6 templateUrl: './sidebar.component.html',
7 providers: [SubscribeService] //2. WHAT TO PROVIDE
8 })
9 export class SidebarComponent {
10
11 //1. HOW TO PROVIDE DEPENDENCY
12 constructor(private subService: SubscribeService){
13
14 }
15
16 OnSubscribe(){
17 this.subService.OnSubscribeClicked('quaterly');
18 }
19 }
20

Dependency injection (DI) is a technique (design pattern) using which a class receives its dependencies from an external source rather than creating them itself.

Advantage of using Dependency Injection (DI)

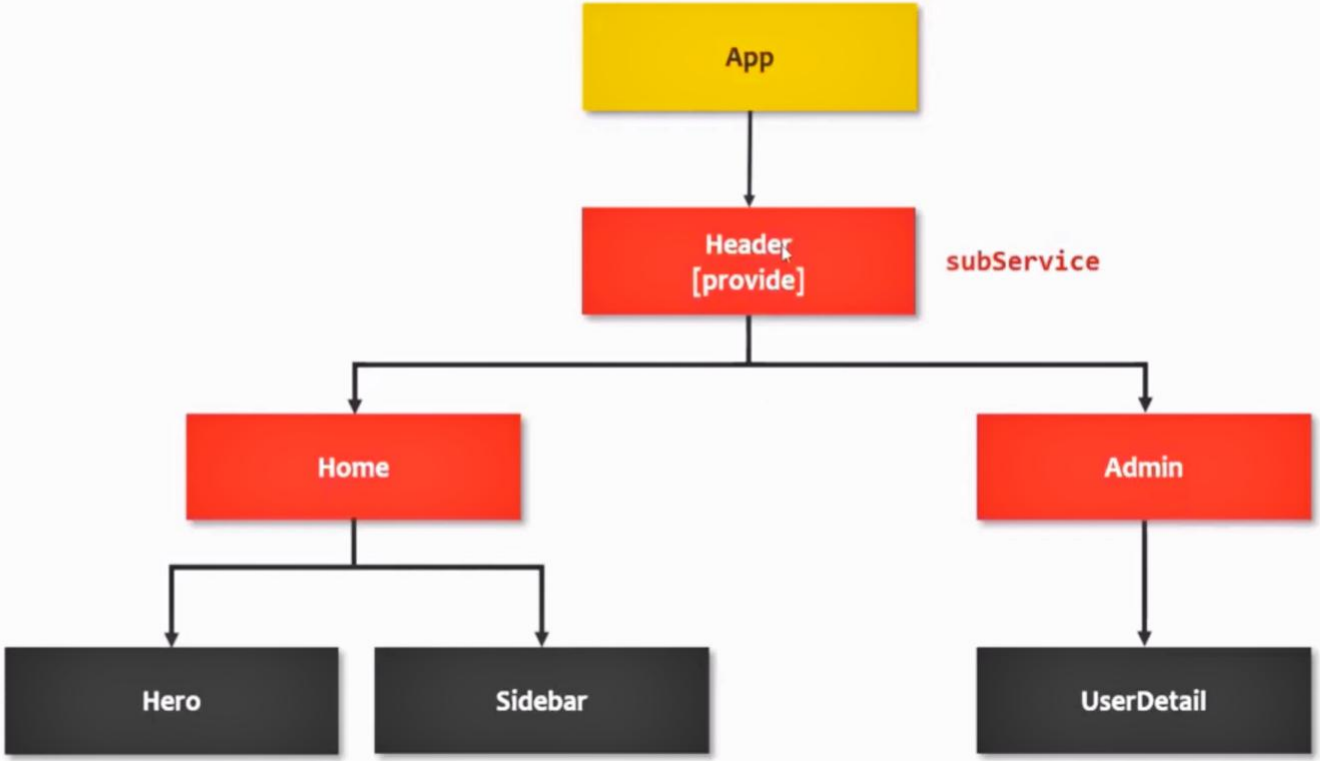
1. Dependency injection or DI keeps the code flexible, testable, and mutable
2. Classes can inherit external logic without knowing how to create it.
3. Dependency injection benefits components, directives and pipes.

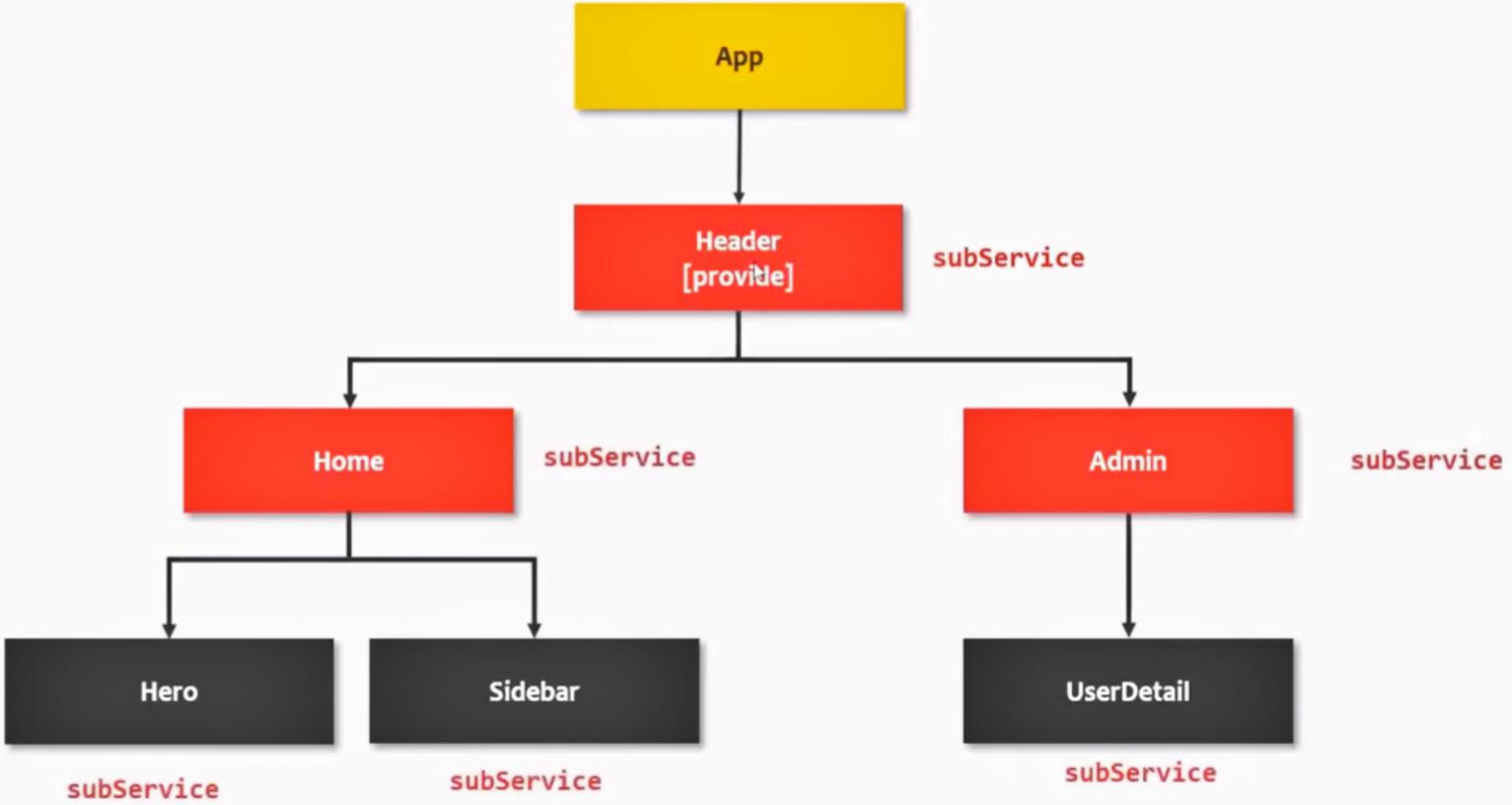
1 When we provide a service on multiple components, each component get its own instance of that service.



Hierarchical Injection

When we provide a dependency on a component, the same instance of that dependency is injected in component class and all its child components and their child components. This is called as hierarchical injection.

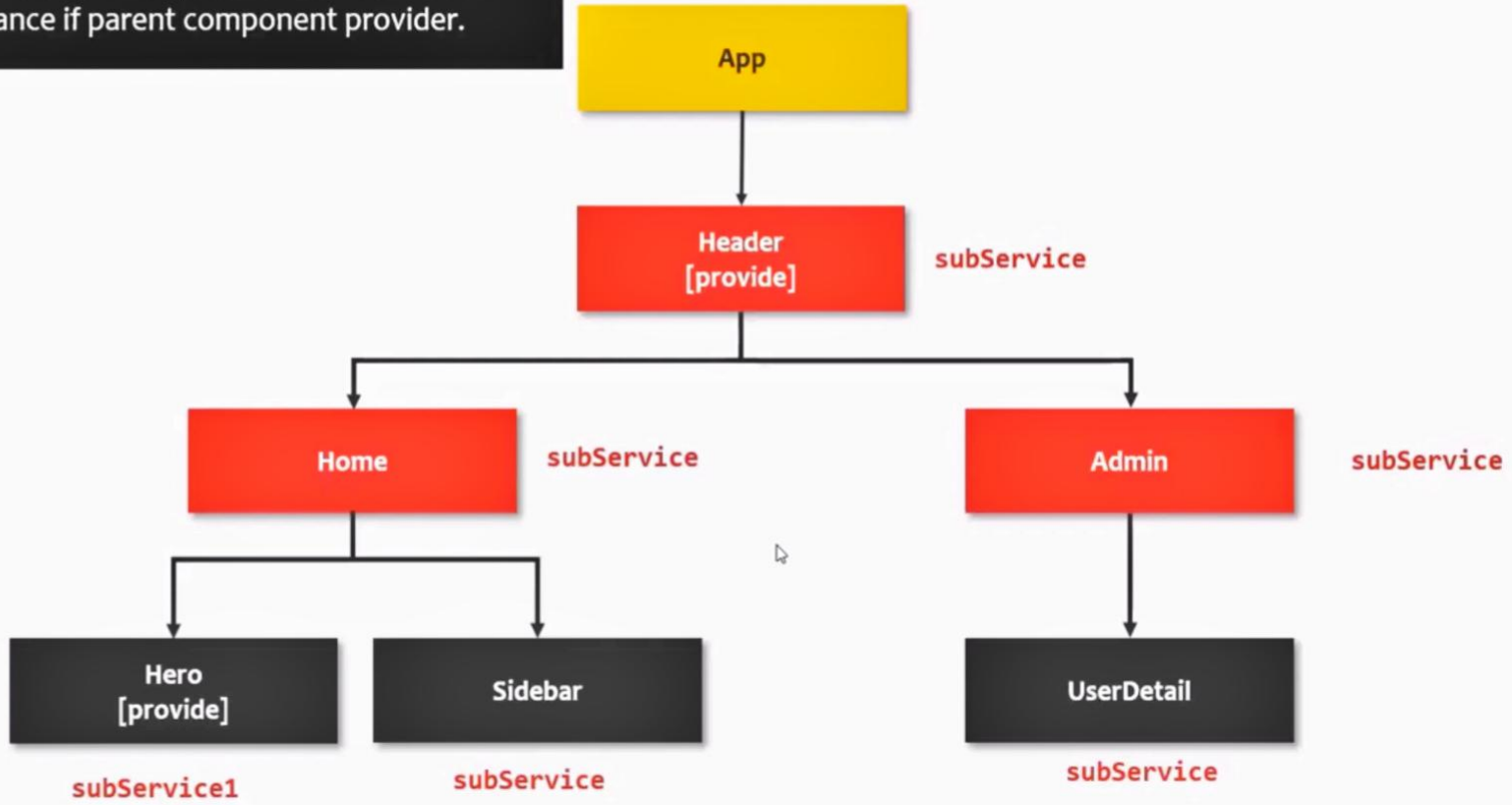

App
Header
[provide]
subService
Home
Admin
Hero
Sidebar
UserDetail


App
Header
[provide]
subService
Home
subService
Admin
subService
Hero
Sidebar
UserDetail
subService
subService
subService

Dependency Override

When we provide a dependency on a component and we also provide a dependency on its child component, child component dependency instance will override its parent component dependency instance.

3 Child component provider overrides the instance if parent component provider.



Dependency injection on Root Component

When we provide a dependency on root component, same instance of that dependency is injected to all components, directives and services.

File Edit Selection View Go ··· angular-services-dependency-injection

EXPLORER

OPEN EDITORS

ANGULAR-SERVICES-DEPENDENCY...

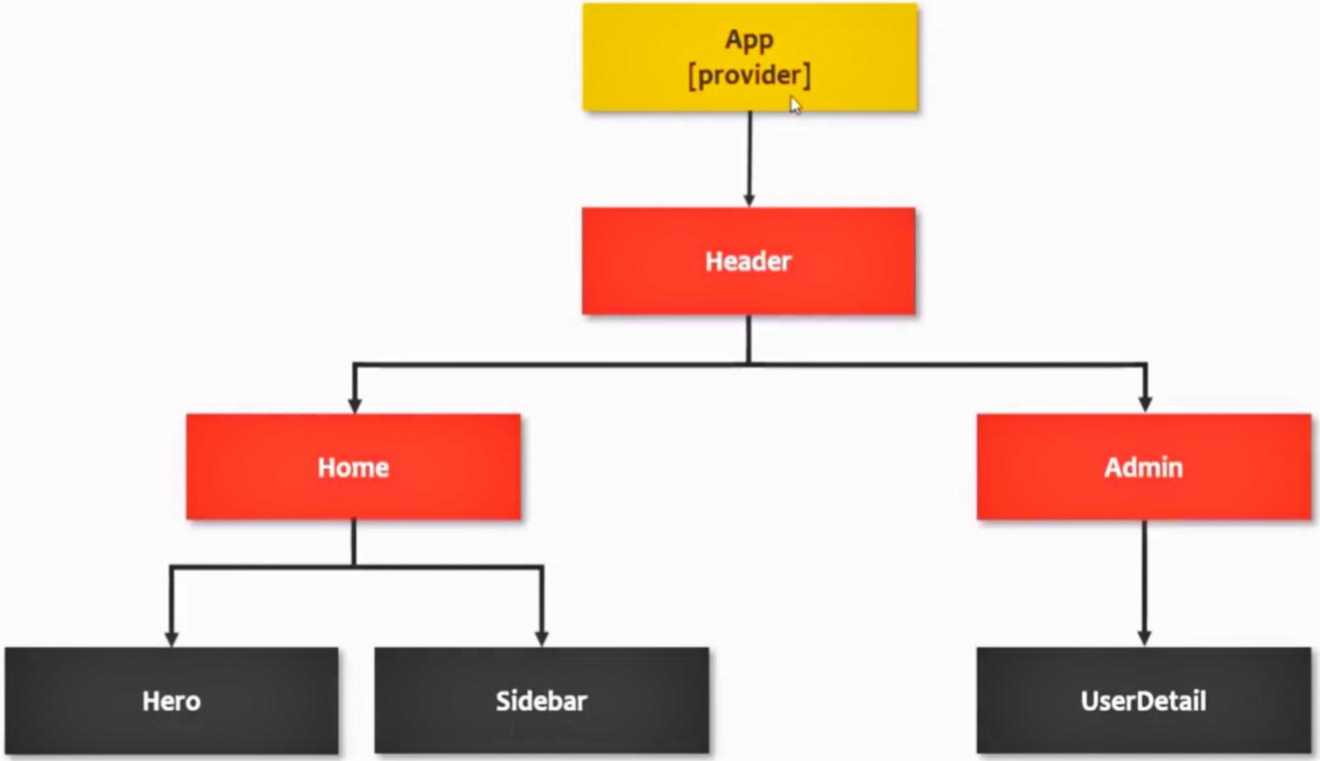
- .angular
- .vscode
- node_modules
- src
  - app
    - header
      - admin
      - home
        - hero
          - hero.component.html
          - hero.component.ts
        - sidebar
          - sidebar.component.html
          - sidebar.component.ts
        - home.component.html
        - home.component.ts
      - header.component.html
      - header.component.ts
    - Services
      - subscribe.service.ts

TIMELINE

subscribe.service.ts | header.component.ts | hero.component.ts | sidebar.component.ts

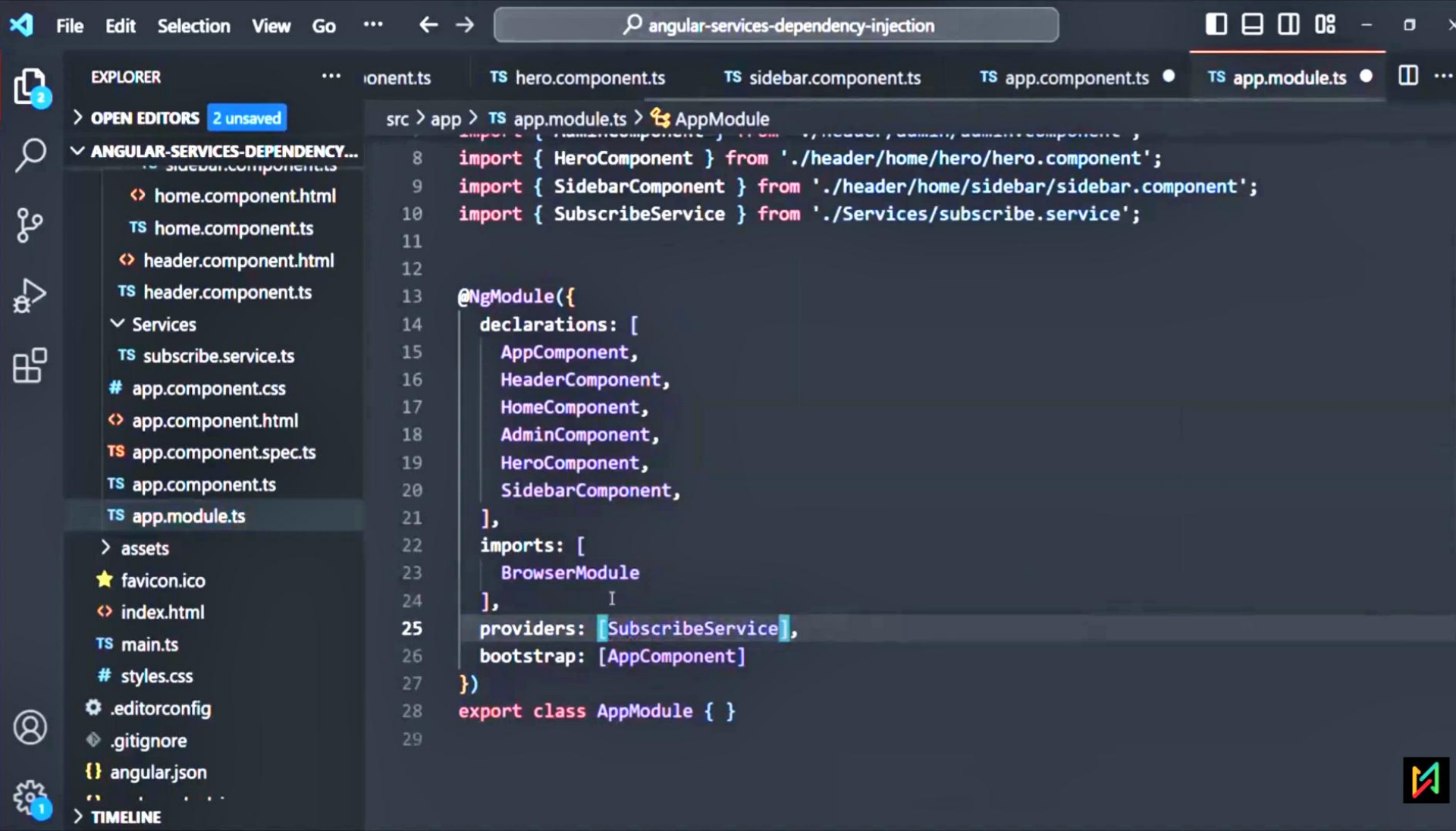
src > app > header > header.component.ts > HeaderComponent > constructor

```ts
import { Component } from '@angular/core';
import { SubscribeService } from '../Services/subscribe.service';

@Component({
  selector: 'app-header',
  templateUrl: './header.component.html',
  providers: [SubscribeService] //2. WHAT TO PROVIDE
})
export class HeaderComponent {
  selectedTab: string = 'home';

  //1. HOW TO PROVIDE DEPENDENCY
  constructor(private subService: SubscribeService){

  }

  //When HOME Link is clicked
  HomeClicked(){
    this.selectedTab = 'home';
  }

  //When Admin Link is clicked
  AdminClicked(){
    this.selectedTab = 'admin';
  }
```

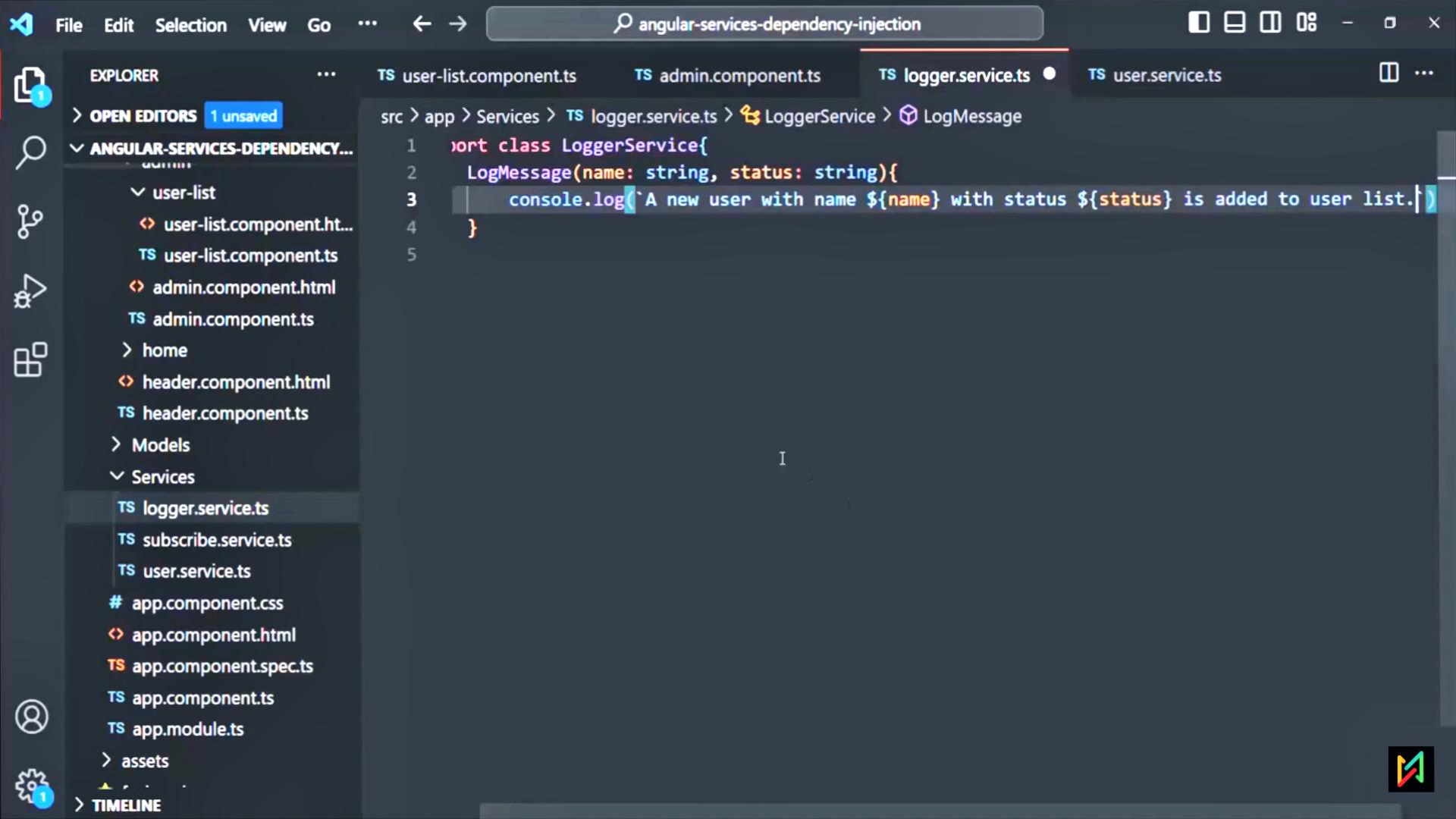

App
[provider]
Header
Home
Admin
Hero
Sidebar
UserDetail

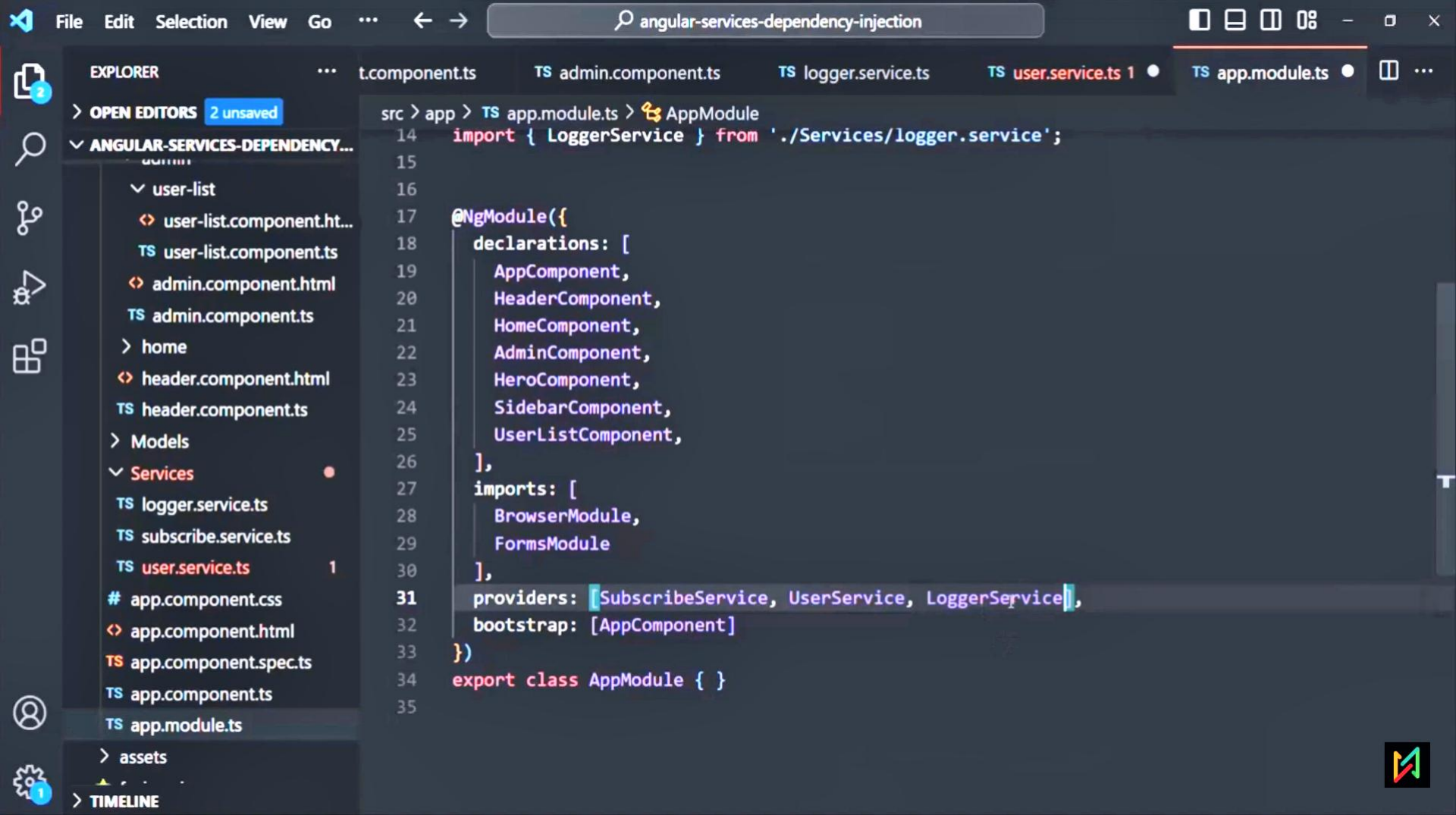
Module Injector

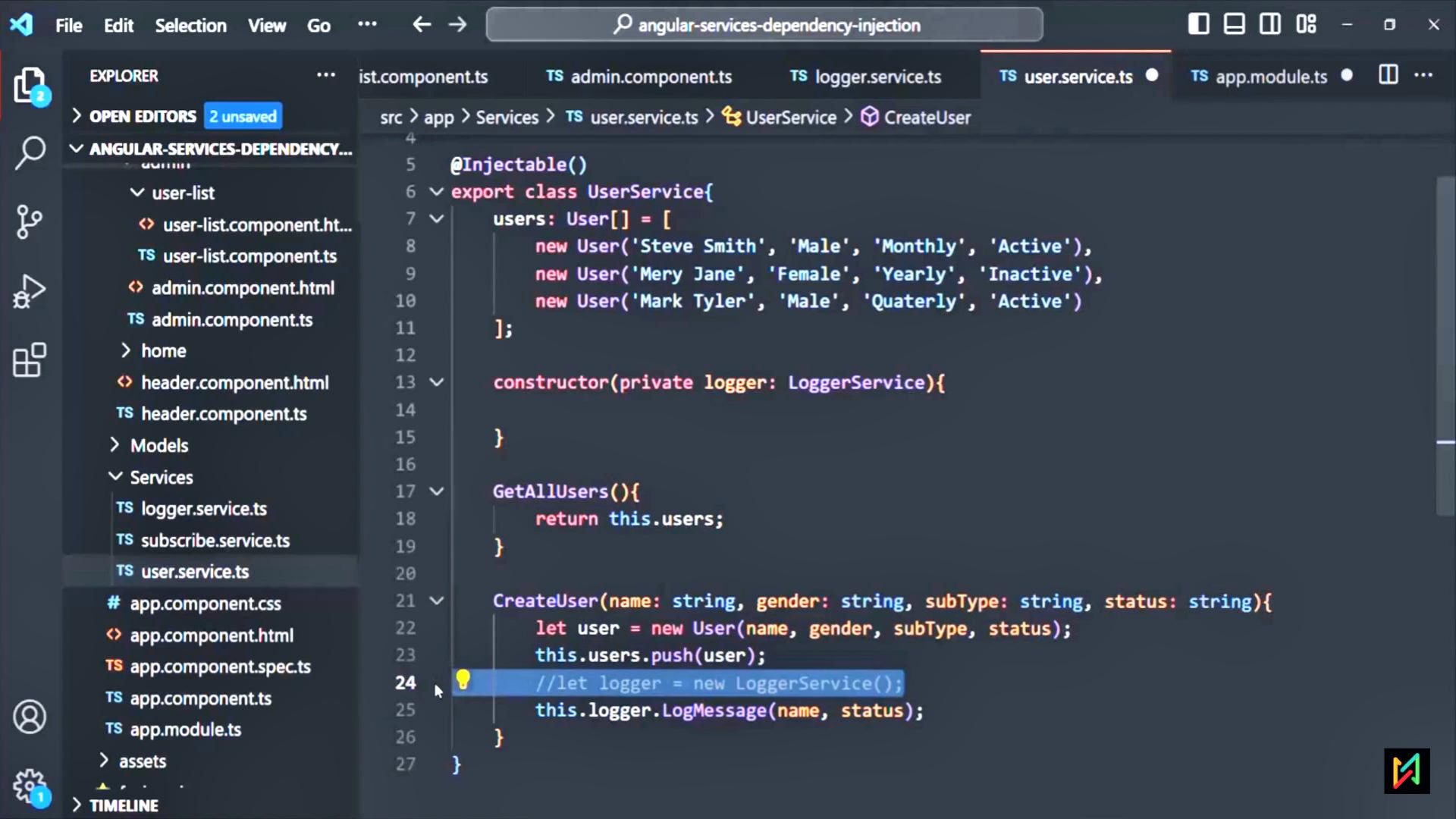
We can also inject a service from Module class. In that case same instance of the dependency will be available throughout the Angular application. In this way we implement singleton pattern where a single instance is shared throughout the application.

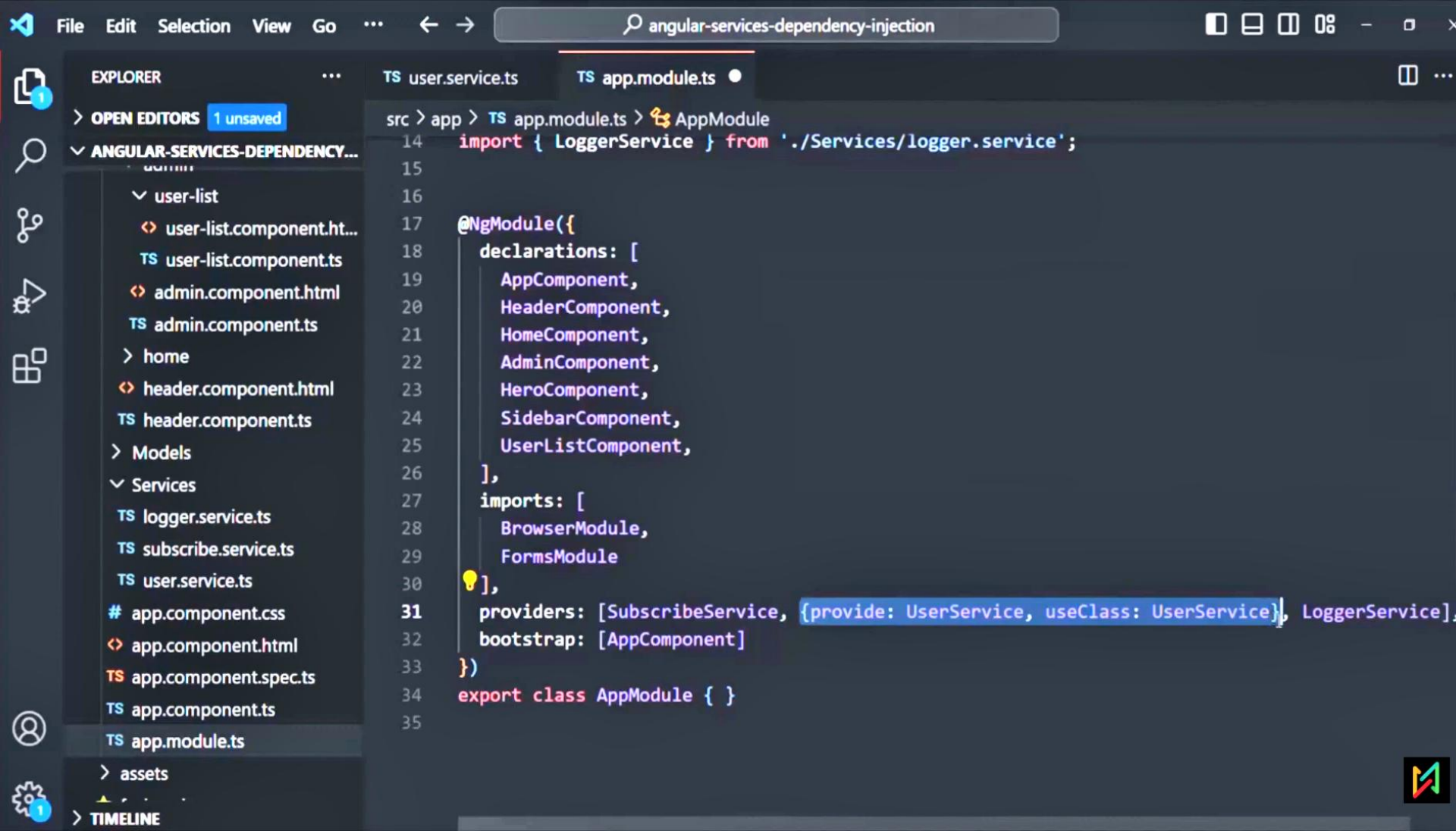

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
OPEN EDITORS 2 unsaved
ANGULAR-SERVICES-DEPENDENCY...
home.component.html
home.component.ts
header.component.html
header.component.ts
Services
subscribe.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
favicon.ico
index.html
main.ts
styles.css
.editorconfig
.gitignore
angular.json
TIMELINE
hero.component.ts
sidebar.component.ts
app.component.ts
app.module.ts
src > app > app.module.ts > AppModule
import { HeroComponent } from './header/home/hero/hero.component';
import { SidebarComponent } from './header/home/sidebar/sidebar.component';
import { SubscribeService } from './Services/subscribe.service';
@NgModule({
declarations: [
AppComponent,
HeaderComponent,
HomeComponent,
AdminComponent,
HeroComponent,
SidebarComponent,
],
imports: [
BrowserModule
],
providers: [SubscribeService],
bootstrap: [AppComponent]
})
export class AppModule { }

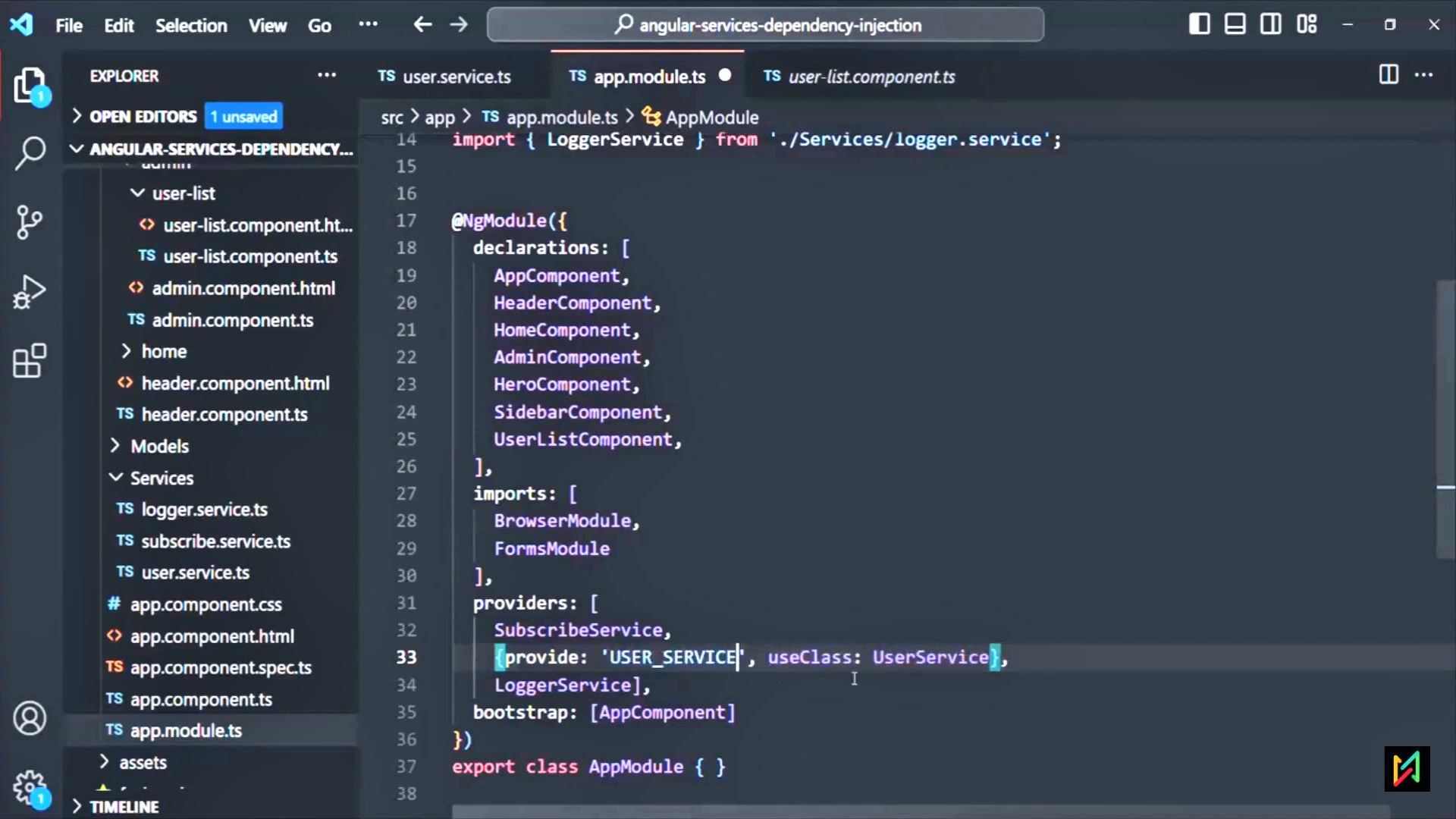
Injecting Service into Another Service

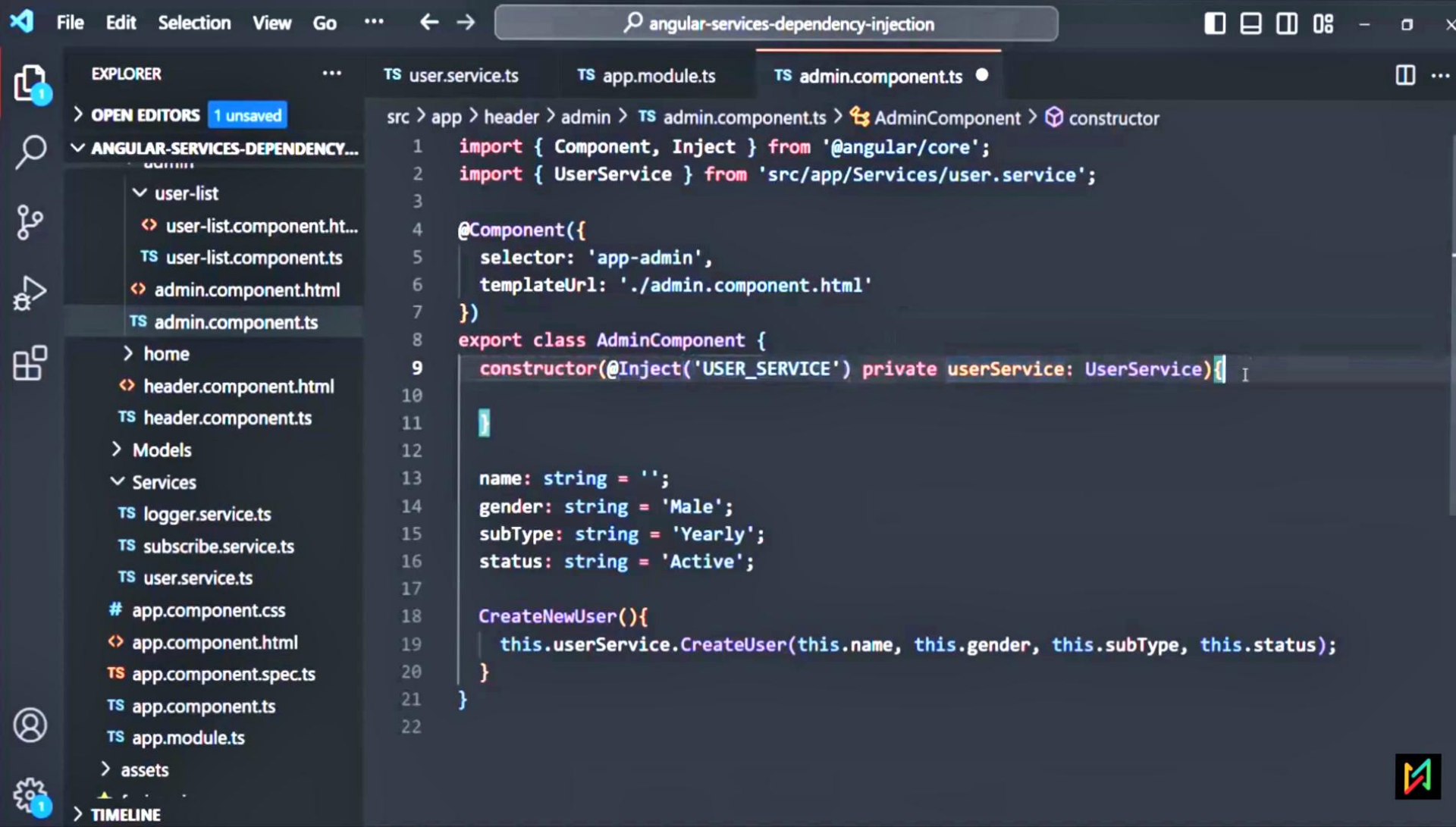

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
user-list.component.ts
admin.component.ts
logger.service.ts
user.service.ts
OPEN EDITORS 1 unsaved
ANGULAR-SERVICES-DEPENDENCY...
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
TIMELINE
src > app > Services > logger.service.ts > LoggerService > LogMessage
ort class LoggerService{
LogMessage(name: string, status: string){
console.log(`A new user with name ${name} with status ${status} is added to user list.`)
}

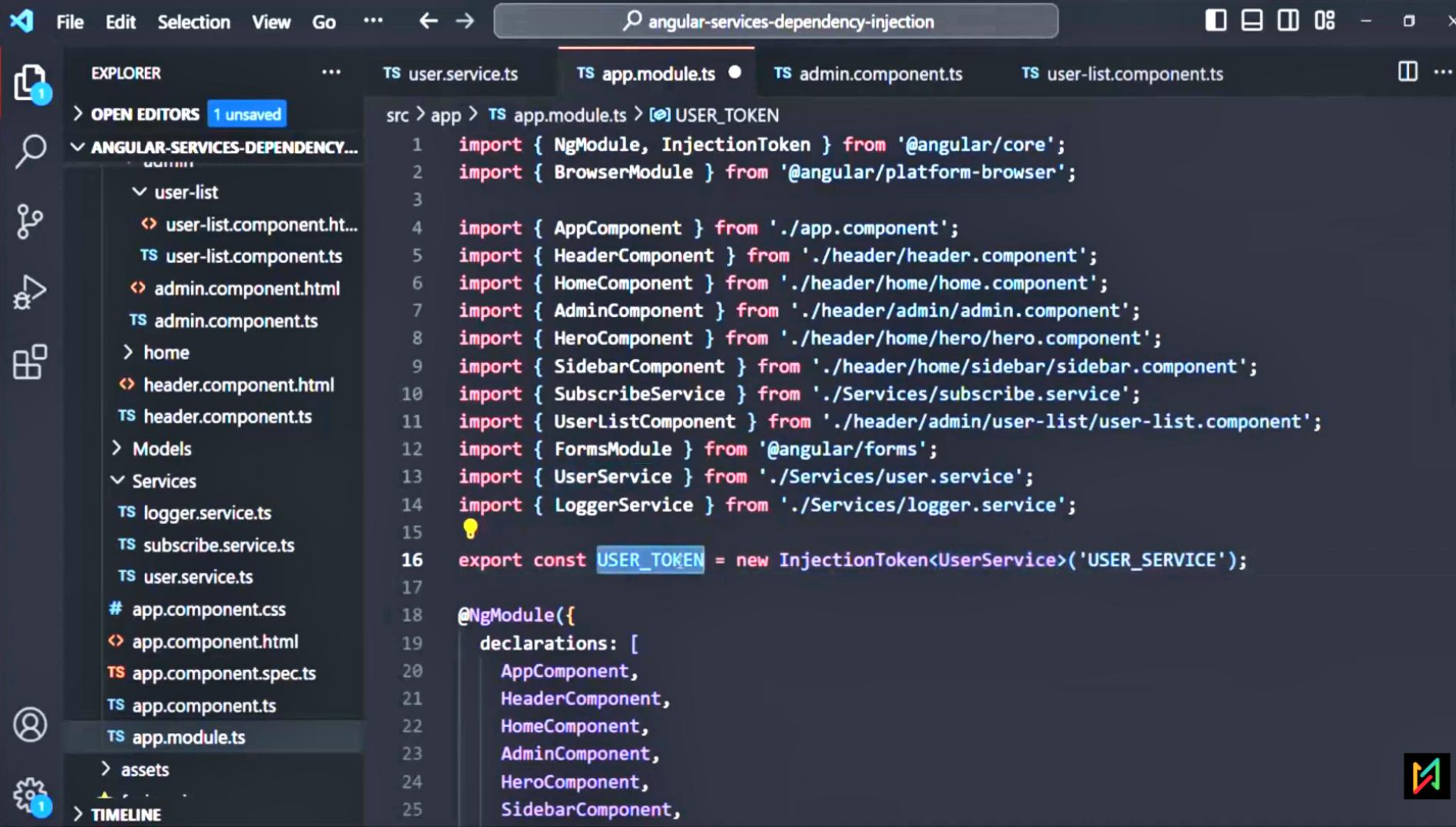

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
t.component.ts
admin.component.ts
logger.service.ts
user.service.ts 1
app.module.ts
OPEN EDITORS 2 unsaved
ANGULAR-SERVICES-DEPENDENCY...
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts 1
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
TIMELINE
src > app > app.module.ts > AppModule
14 import { LoggerService } from './Services/logger.service';
15
16
17 @NgModule({
18 declarations: [
19 AppComponent,
20 HeaderComponent,
21 HomeComponent,
22 AdminComponent,
23 HeroComponent,
24 SidebarComponent,
25 UserListComponent,
26],
27 imports: [
28 BrowserModule,
29 FormsModule
30],
31 providers: [SubscribeService, UserService, LoggerService],
32 bootstrap: [AppComponent]
33 })
34 export class AppModule { }
35

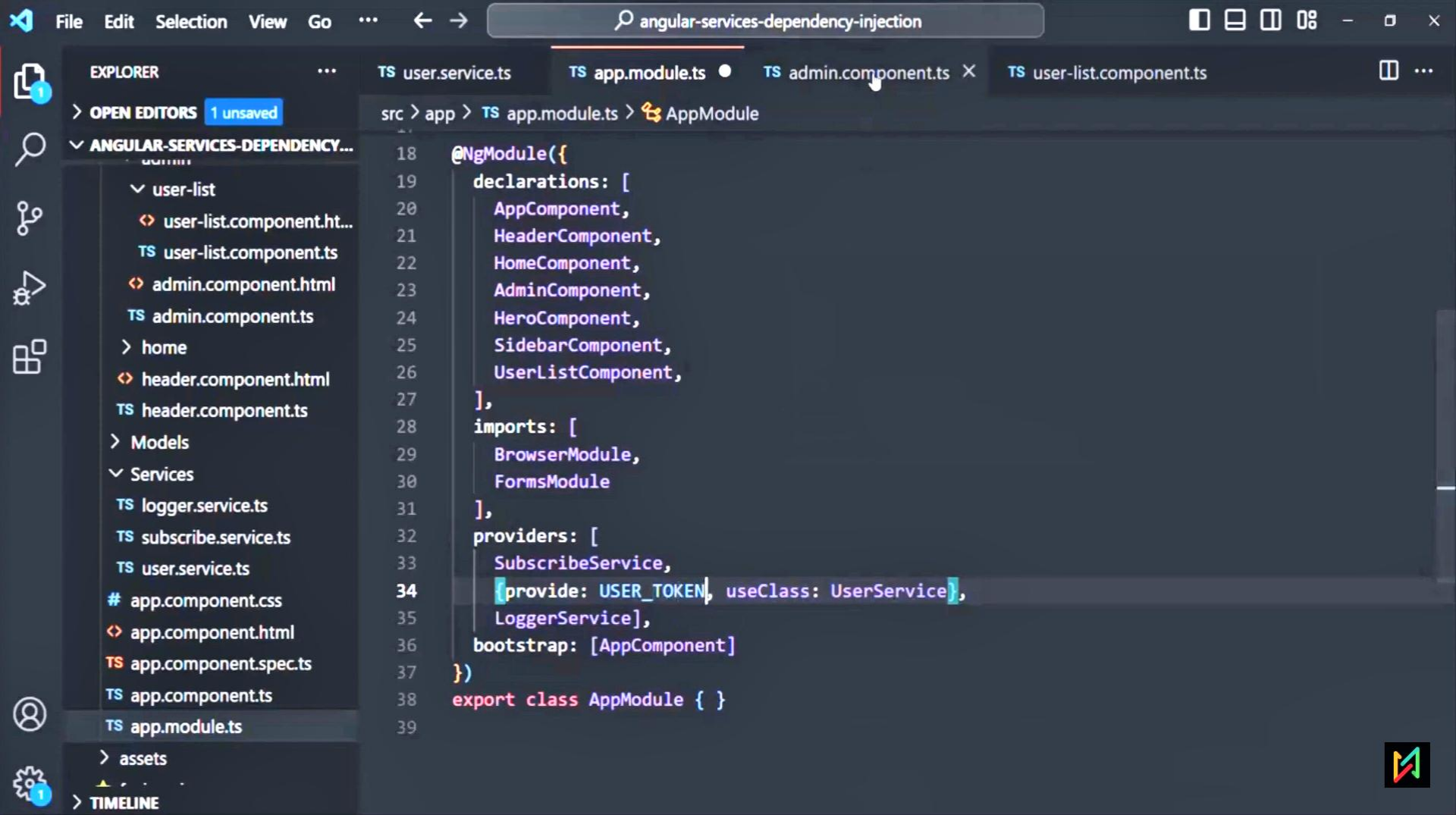

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
OPEN EDITORS 2 unsaved
ANGULAR-SERVICES-DEPENDENCY...
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
TIMELINE
ist.component.ts
admin.component.ts
logger.service.ts
user.service.ts
app.module.ts
src > app > Services > user.service.ts > UserService > CreateUser
@Injectable()
export class UserService{
users: User[] = [
new User('Steve Smith', 'Male', 'Monthly', 'Active'),
new User('Mery Jane', 'Female', 'Yearly', 'Inactive'),
new User('Mark Tyler', 'Male', 'Quaterly', 'Active')
];
constructor(private logger: LoggerService){
}
GetAllUsers(){
return this.users;
}
CreateUser(name: string, gender: string, subType: string, status: string){
let user = new User(name, gender, subType, status);
this.users.push(user);
//let logger = new LoggerService();
this.logger.LogMessage(name, status);
}
}

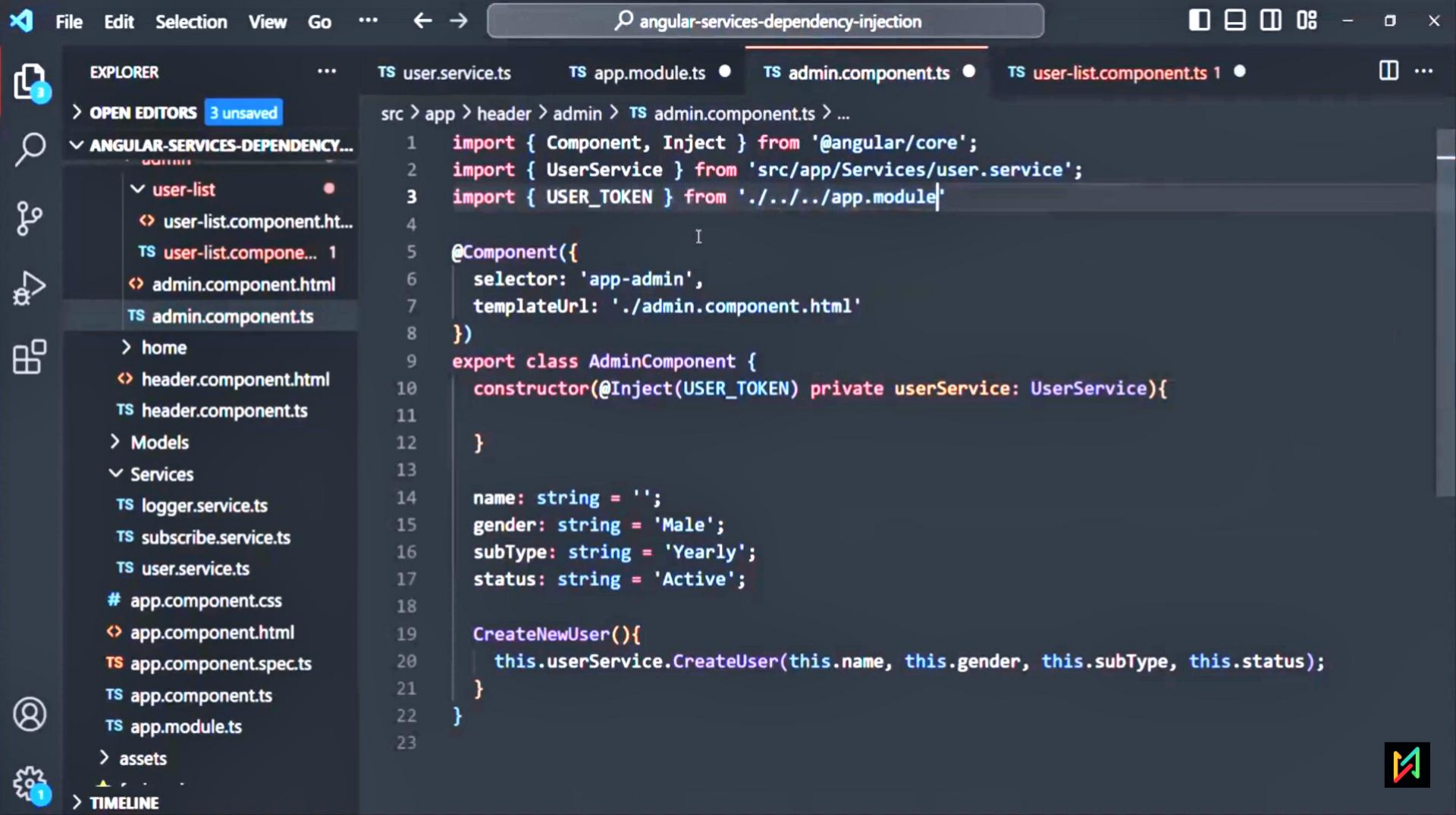

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
OPEN EDITORS 1 unsaved
ANGULAR-SERVICES-DEPENDENCY...
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
TIMELINE
user.service.ts
app.module.ts
src > app > app.module.ts > AppModule
14 import { LoggerService } from './Services/logger.service';
15
16
17 @NgModule({
18 declarations: [
19 AppComponent,
20 HeaderComponent,
21 HomeComponent,
22 AdminComponent,
23 HeroComponent,
24 SidebarComponent,
25 UserListComponent,
26],
27 imports: [
28 BrowserModule,
29 FormsModule
30],
31 providers: [SubscribeService, {provide: UserService, useClass: UserService}, LoggerService],
32 bootstrap: [AppComponent]
33 })
34 export class AppModule { }
35

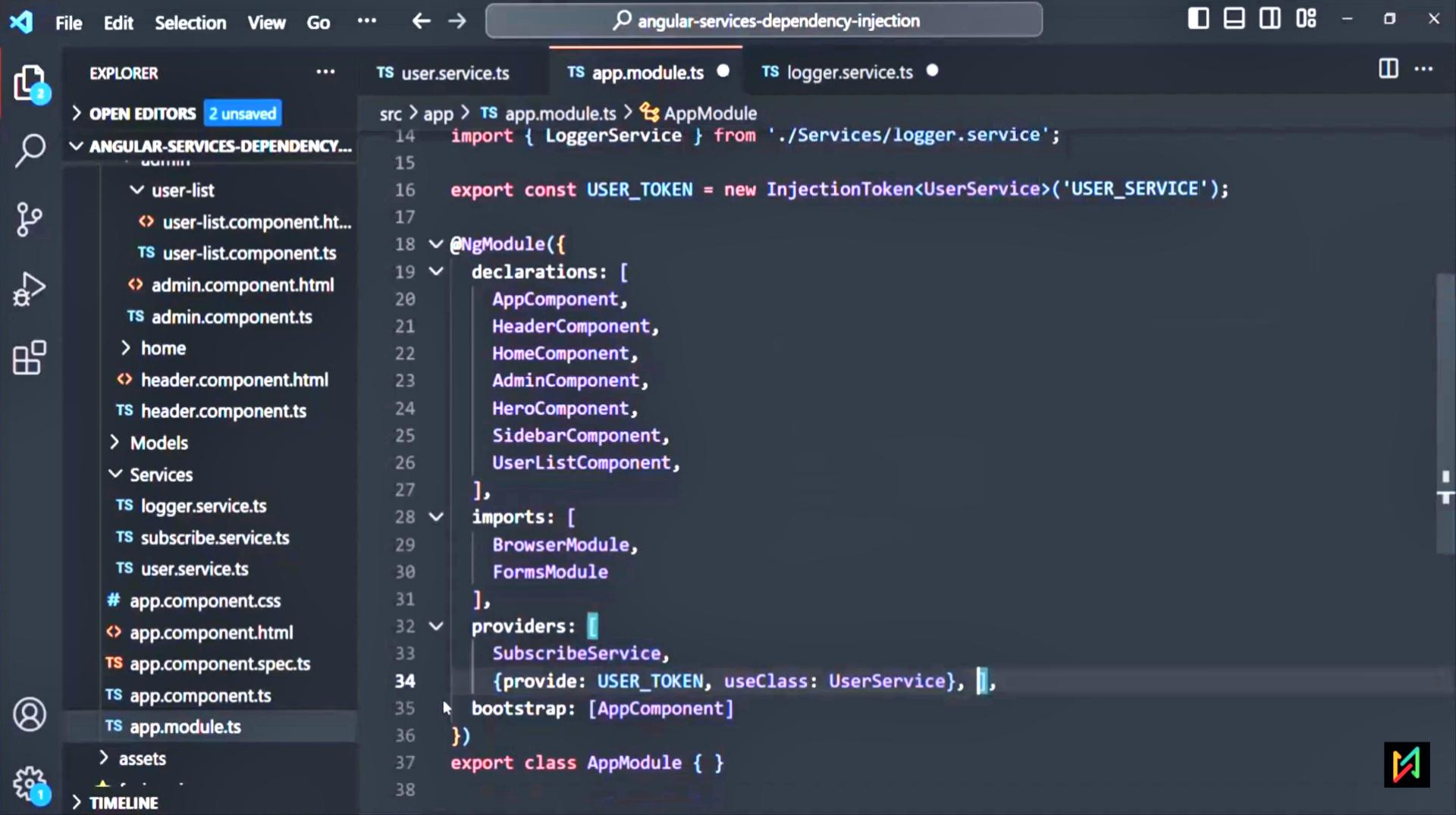

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
OPEN EDITORS 1 unsaved
ANGULAR-SERVICES-DEPENDENCY...
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
TIMELINE
user.service.ts
app.module.ts
user-list.component.ts
src > app > app.module.ts > AppModule
14 import { LoggerService } from './Services/logger.service';
15
16
17 @NgModule({
18 declarations: [
19 AppComponent,
20 HeaderComponent,
21 HomeComponent,
22 AdminComponent,
23 HeroComponent,
24 SidebarComponent,
25 UserListComponent,
26],
27 imports: [
28 BrowserModule,
29 FormsModule
30],
31 providers: [
32 SubscribeService,
33 {provide: 'USER_SERVICE', useClass: UserService},
34 LoggerService],
35 bootstrap: [AppComponent]
36 })
37 export class AppModule { }
38

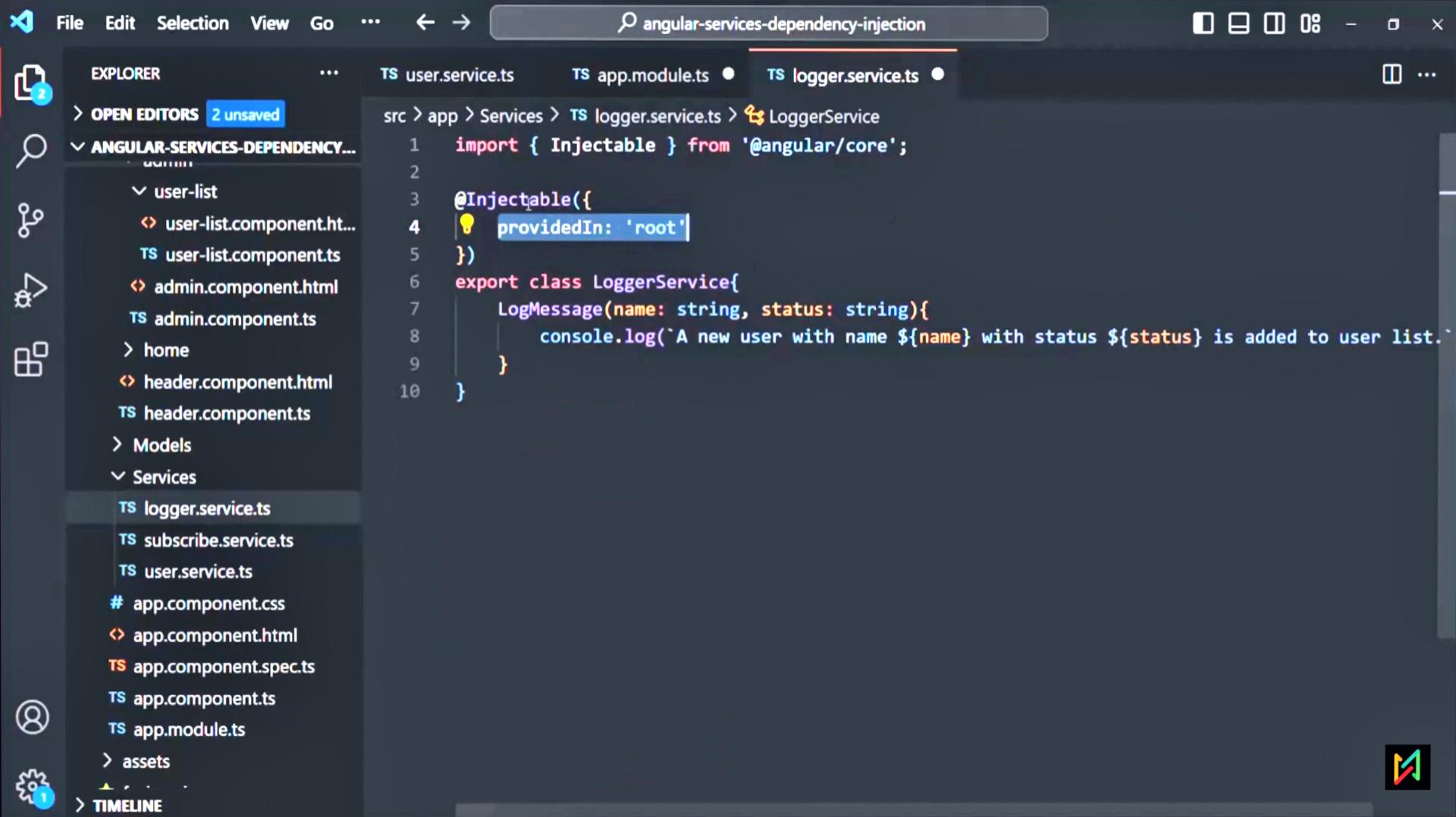

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
OPEN EDITORS 1 unsaved
ANGULAR-SERVICES-DEPENDENCY...
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
TIMELINE
user.service.ts
app.module.ts
admin.component.ts
src > app > header > admin > admin.component.ts > AdminComponent > constructor
1 import { Component, Inject } from '@angular/core';
2 import { UserService } from 'src/app/Services/user.service';
3
4 @Component({
5 selector: 'app-admin',
6 templateUrl: './admin.component.html'
7 })
8 export class AdminComponent {
9 constructor(@Inject('USER_SERVICE') private userService: UserService){
10
11 }
12
13 name: string = '';
14 gender: string = 'Male';
15 subType: string = 'Yearly';
16 status: string = 'Active';
17
18 CreateNewUser(){
19 this.userService.CreateUser(this.name, this.gender, this.subType, this.status);
20 }
21 }
22

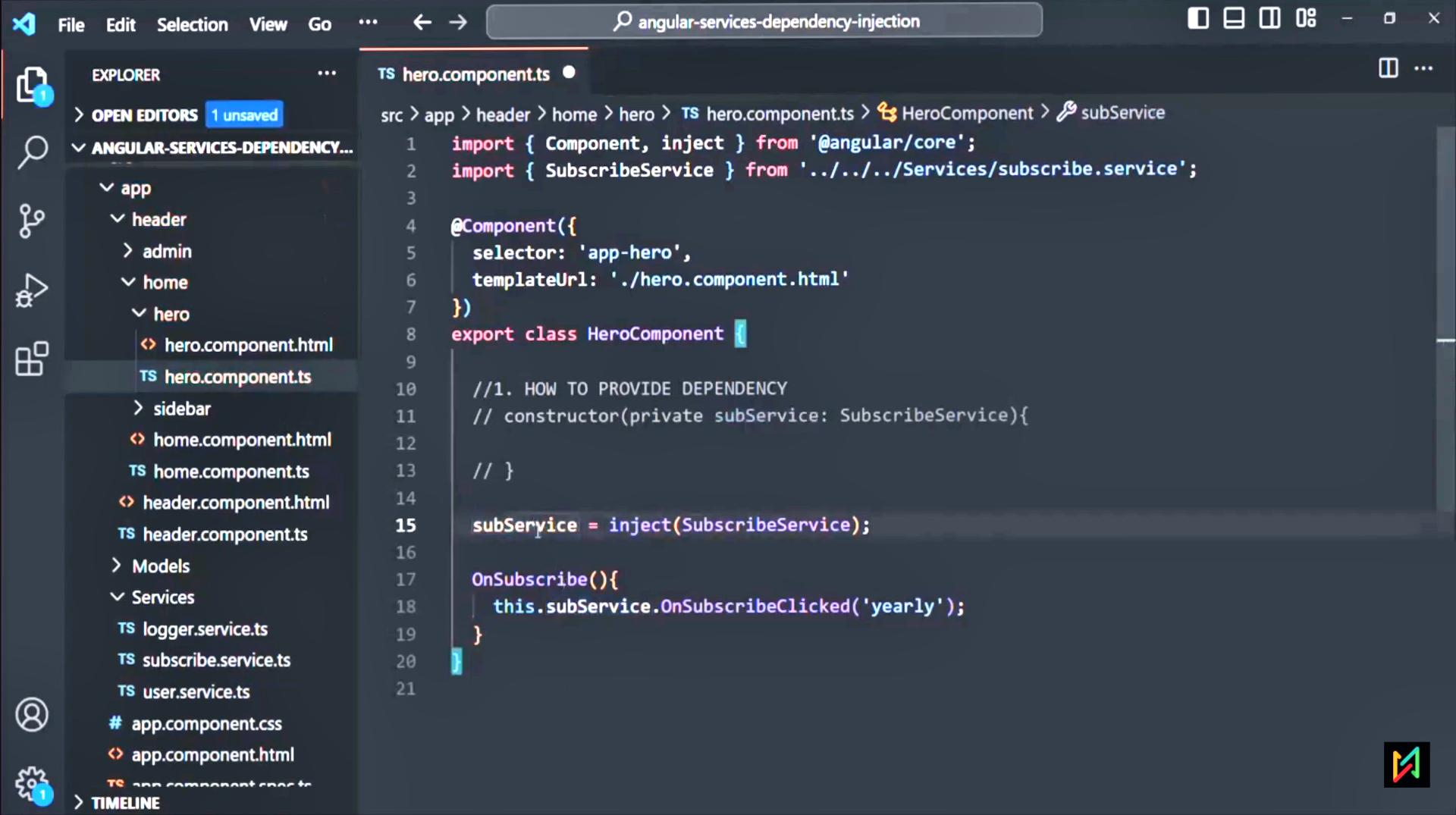

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
OPEN EDITORS 1 unsaved
ANGULAR-SERVICES-DEPENDENCY...
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
TIMELINE
user.service.ts
app.module.ts
admin.component.ts
user-list.component.ts
src > app > app.module.ts > USER_TOKEN
1 import { NgModule, InjectionToken } from '@angular/core';
2 import { BrowserModule } from '@angular/platform-browser';
3
4 import { AppComponent } from './app.component';
5 import { HeaderComponent } from './header/header.component';
6 import { HomeComponent } from './header/home/home.component';
7 import { AdminComponent } from './header/admin/admin.component';
8 import { HeroComponent } from './header/home/hero/hero.component';
9 import { SidebarComponent } from './header/home/sidebar/sidebar.component';
10 import { SubscribeService } from './Services/subscribe.service';
11 import { UserListComponent } from './header/admin/user-list/user-list.component';
12 import { FormsModule } from '@angular/forms';
13 import { UserService } from './Services/user.service';
14 import { LoggerService } from './Services/logger.service';
15
16 export const USER_TOKEN = new InjectionToken<UserService>('USER_SERVICE');
17
18 @NgModule({
19 declarations: [
20 AppComponent,
21 HeaderComponent,
22 HomeComponent,
23 AdminComponent,
24 HeroComponent,
25 SidebarComponent,


File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
OPEN EDITORS 1 unsaved
ANGULAR-SERVICES-DEPENDENCY...
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
TIMELINE
user.service.ts
app.module.ts
admin.component.ts
user-list.component.ts
src > app > app.module.ts > AppModule
@NgModule({
declarations: [
AppComponent,
HeaderComponent,
HomeComponent,
AdminComponent,
HeroComponent,
SidebarComponent,
UserListComponent,
],
imports: [
BrowserModule,
FormsModule
],
providers: [
SubscribeService,
{provide: USER_TOKEN, useClass: UserService},
LoggerService],
bootstrap: [AppComponent]
})
export class AppModule { }

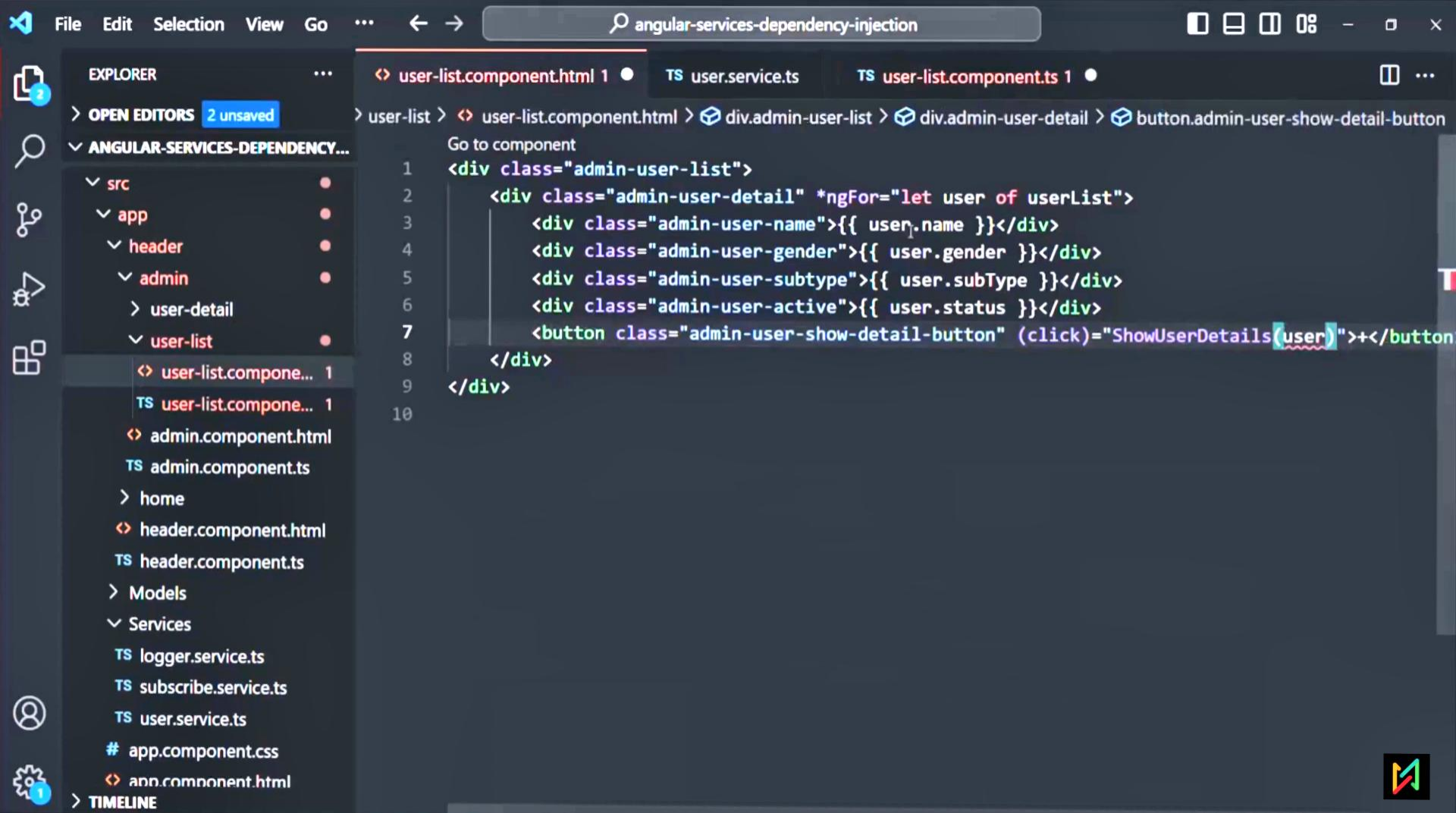

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
OPEN EDITORS 3 unsaved
ANGULAR-SERVICES-DEPENDENCY...
user-list
user-list.component.ht...
user-list.compone... 1
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
TIMELINE
user.service.ts
app.module.ts
admin.component.ts
user-list.component.ts 1
src > app > header > admin > admin.component.ts > ...
1 import { Component, Inject } from '@angular/core';
2 import { UserService } from 'src/app/Services/user.service';
3 import { USER_TOKEN } from './../../app.module'
4
5 @Component({
6 selector: 'app-admin',
7 templateUrl: './admin.component.html'
8 })
9 export class AdminComponent {
10 constructor(@Inject(USER_TOKEN) private userService: UserService){
11
12 }
13
14 name: string = '';
15 gender: string = 'Male';
16 subType: string = 'Yearly';
17 status: string = 'Active';
18
19 CreateNewUser(){
20 this.userService.CreateUser(this.name, this.gender, this.subType, this.status);
21 }
22 }
23

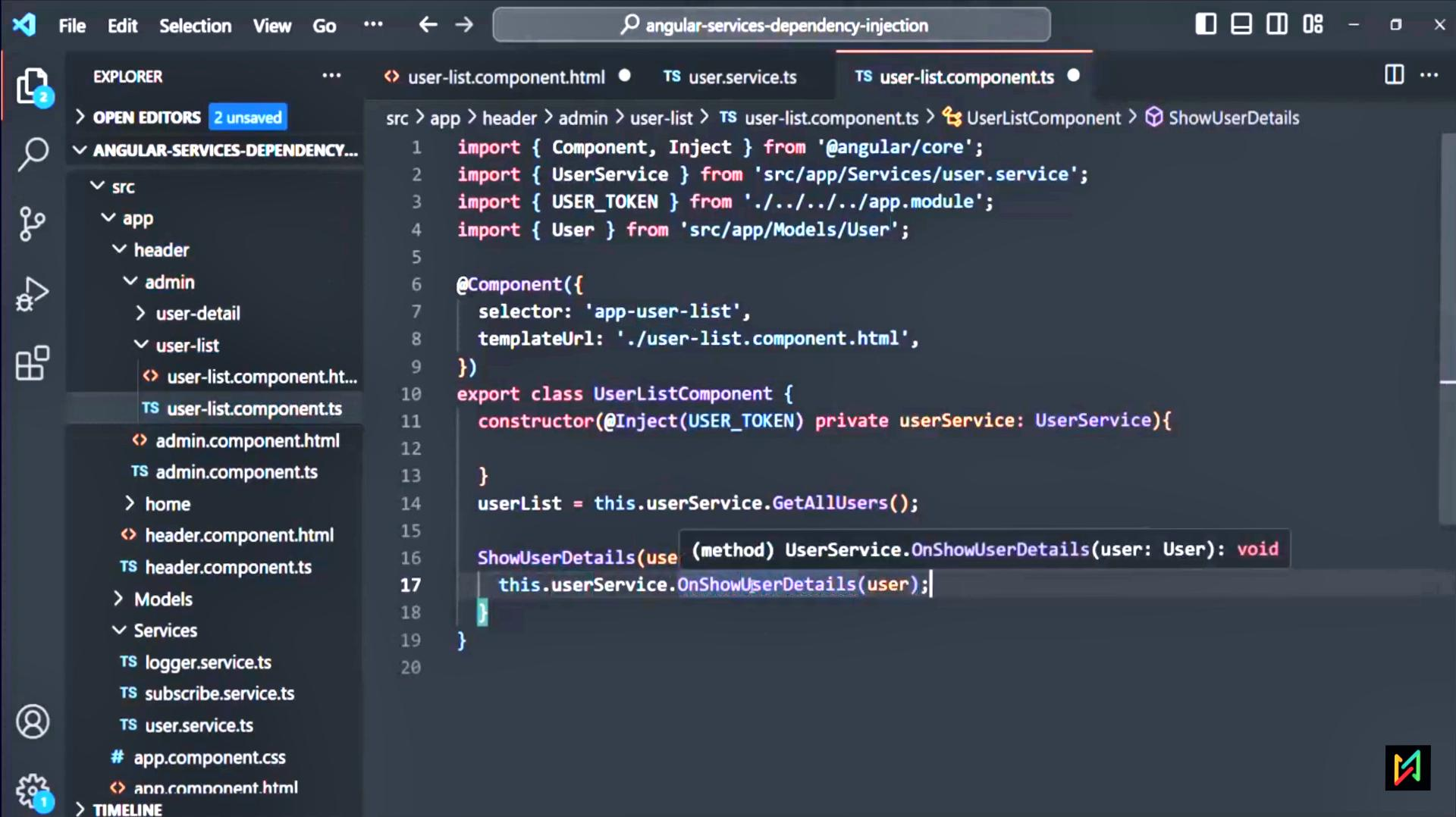

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
user.service.ts
app.module.ts
logger.service.ts
OPEN EDITORS 2 unsaved
ANGULAR-SERVICES-DEPENDENCY...
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
TIMELINE
src > app > app.module.ts > AppModule
14 import { LoggerService } from './Services/logger.service';
15
16 export const USER_TOKEN = new InjectionToken<UserService>('USER_SERVICE');
17
18 @NgModule({
19 declarations: [
20 AppComponent,
21 HeaderComponent,
22 HomeComponent,
23 AdminComponent,
24 HeroComponent,
25 SidebarComponent,
26 UserListComponent,
27],
28 imports: [
29 BrowserModule,
30 FormsModule
31],
32 providers: [
33 SubscribeService,
34 {provide: USER_TOKEN, useClass: UserService},
35 bootstrap: [AppComponent]
36 })
37 export class AppModule { }
38

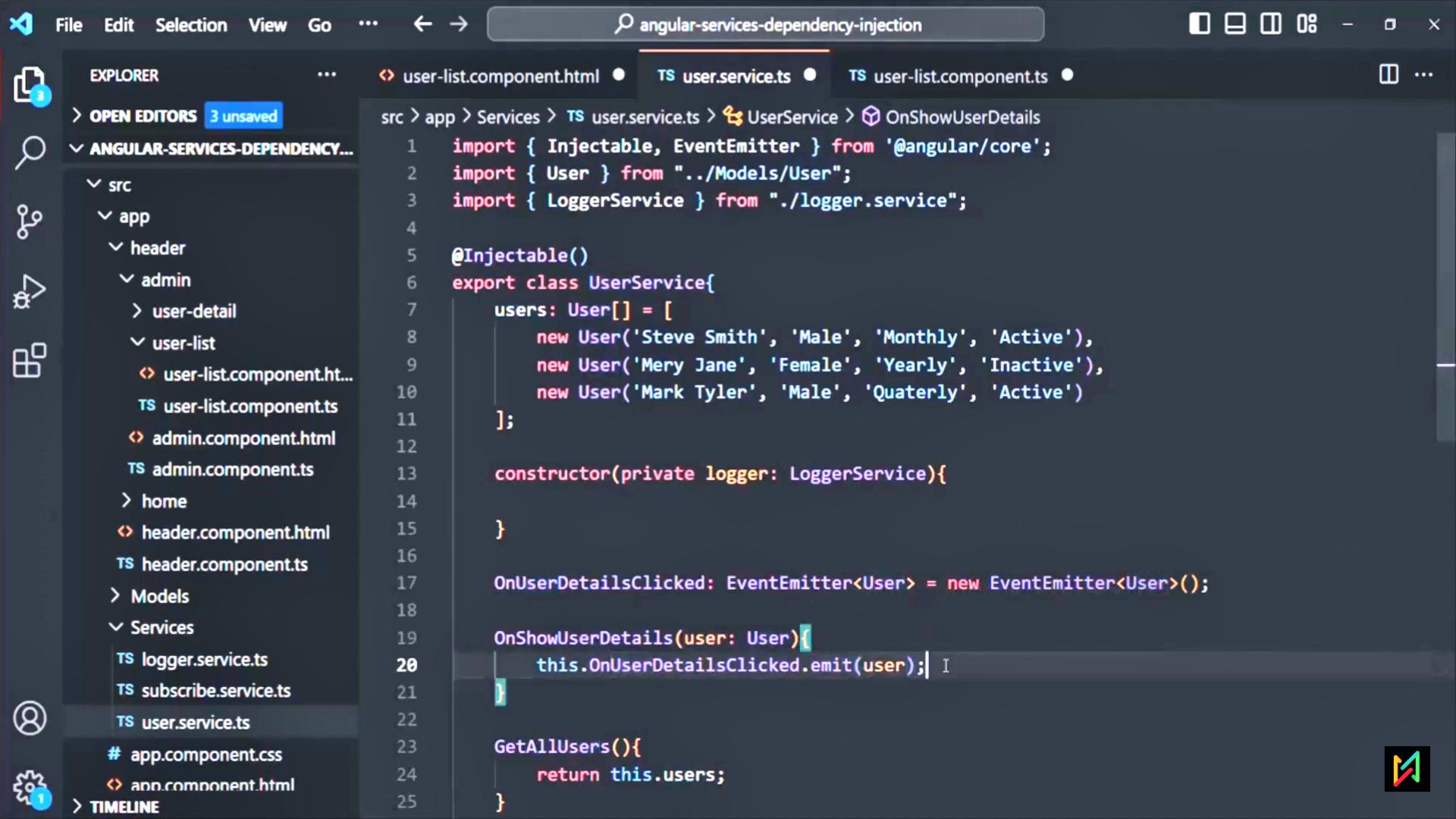

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
user.service.ts
app.module.ts
logger.service.ts
OPEN EDITORS 2 unsaved
ANGULAR-SERVICES-DEPENDENCY...
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
TIMELINE
src > app > Services > logger.service.ts > LoggerService
1 import { Injectable } from '@angular/core';
2
3 @Injectable({
4 providedIn: 'root'
5 })
6 export class LoggerService{
7 LogMessage(name: string, status: string){
8 console.log(`A new user with name ${name} with status ${status} is added to user list.`
9 }
10 }

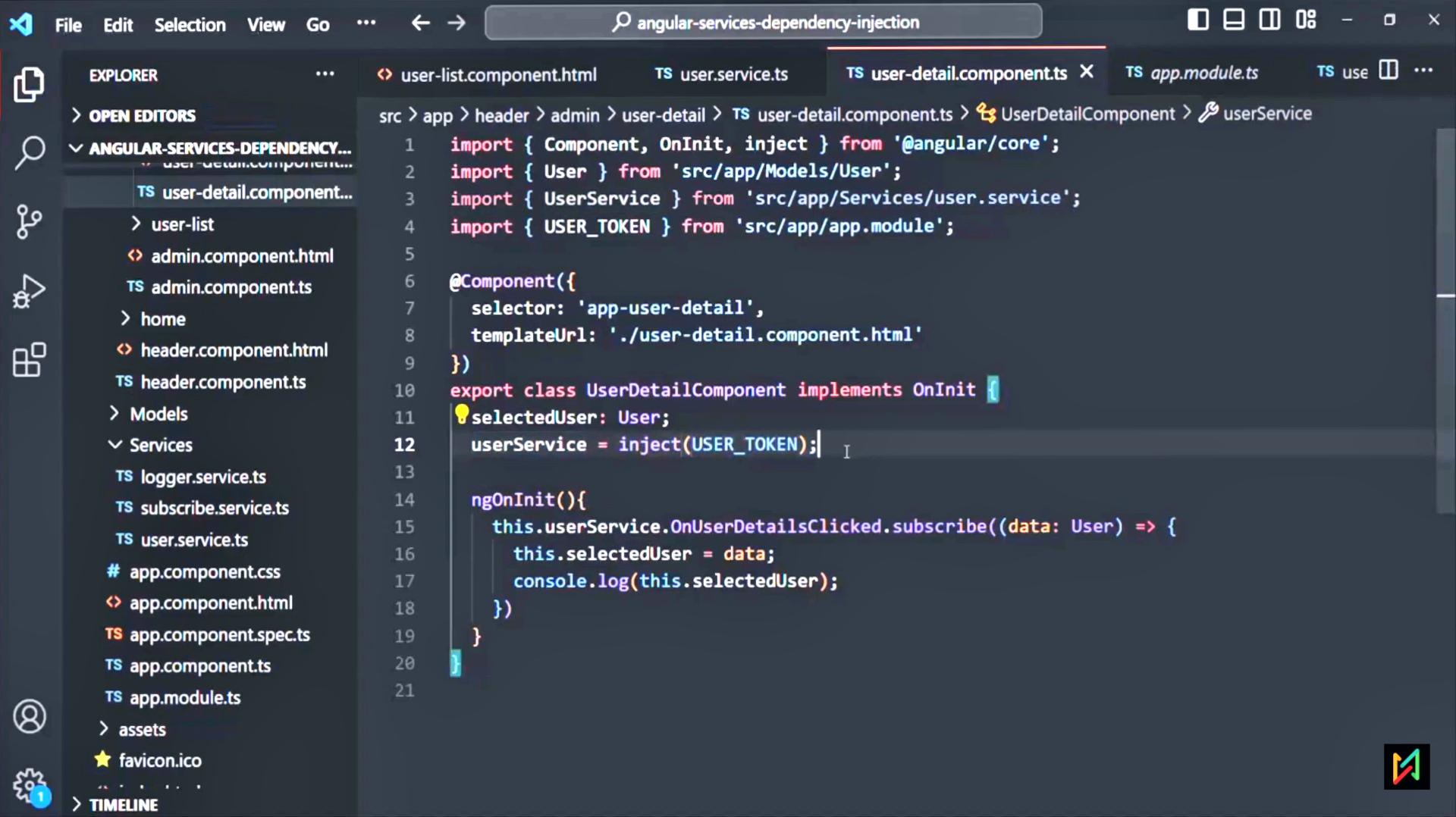

File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
OPEN EDITORS 1 unsaved
ANGULAR-SERVICES-DEPENDENCY...
app
header
admin
home
hero
hero.component.html
hero.component.ts
sidebar
home.component.html
home.component.ts
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
TIMELINE
hero.component.ts
src > app > header > home > hero > hero.component.ts > HeroComponent > subService
1 import { Component, inject } from '@angular/core';
2 import { SubscribeService } from '../../../Services/subscribe.service';
3
4 @Component({
5 selector: 'app-hero',
6 templateUrl: './hero.component.html'
7 })
8 export class HeroComponent {
9
10 //1. HOW TO PROVIDE DEPENDENCY
11 // constructor(private subService: SubscribeService){
12
13 // }
14
15 subService = inject(SubscribeService);
16
17 OnSubscribe(){
18 this.subService.OnSubscribeClicked('yearly');
19 }
20 }
21

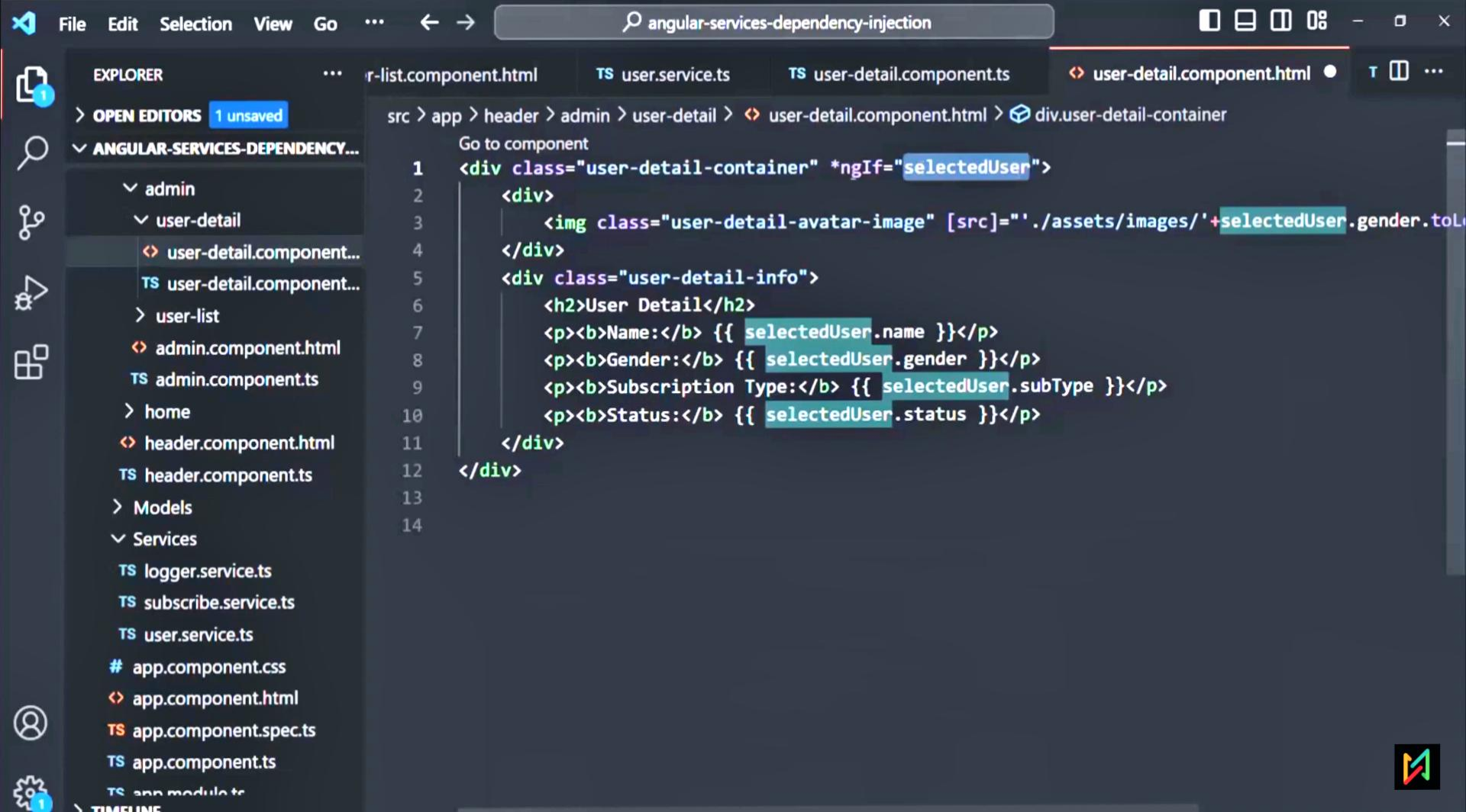
Component Interaction using Services


File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
OPEN EDITORS 2 unsaved
ANGULAR-SERVICES-DEPENDENCY...
src
app
header
admin
user-detail
user-list
user-list.compone... 1
user-list.compone... 1
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
TIMELINE
user-list.component.html 1
user.service.ts
user-list.component.ts 1
user-list > user-list.component.html > div.admin-user-list > div.admin-user-detail > button.admin-user-show-detail-button
Go to component
1
2 <div class="admin-user-detail" *ngFor="let user of userList">
3 {{ user.name }}
4 {{ user.gender }}
5 {{ user.subType }}
6 {{ user.status }}
7 <button class="admin-user-show-detail-button" (click)="ShowUserDetails(user)">+</button
8
9
10


File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
user-list.component.html
user.service.ts
user-list.component.ts
OPEN EDITORS 2 unsaved
ANGULAR-SERVICES-DEPENDENCY...
src
app
header
admin
user-detail
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
TIMELINE
src > app > header > admin > user-list > user-list.component.ts > UserListComponent > ShowUserDetails
import { Component, Inject } from '@angular/core';
import { UserService } from 'src/app/Services/user.service';
import { USER_TOKEN } from './../../../app.module';
import { User } from 'src/app/Models/User';
@Component({
selector: 'app-user-list',
templateUrl: './user-list.component.html',
})
export class UserListComponent {
constructor(@Inject(USER_TOKEN) private userService: UserService){
}
userList = this.userService.GetAllUsers();
ShowUserDetails(use
(method) UserService.OnShowUserDetails(user: User): void
this.userService.OnShowUserDetails(user);
}
}


File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
user-list.component.html
user.service.ts
user-list.component.ts
OPEN EDITORS 3 unsaved
ANGULAR-SERVICES-DEPENDENCY...
src
app
header
admin
user-detail
user-list
user-list.component.ht...
user-list.component.ts
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
TIMELINE
src > app > Services > user.service.ts > UserService > OnShowUserDetails
1 import { Injectable, EventEmitter } from '@angular/core';
2 import { User } from "../Models/User";
3 import { LoggerService } from "./logger.service";
4
5 @Injectable()
6 export class UserService{
7 users: User[] = [
8 new User('Steve Smith', 'Male', 'Monthly', 'Active'),
9 new User('Mery Jane', 'Female', 'Yearly', 'Inactive'),
10 new User('Mark Tyler', 'Male', 'Quaterly', 'Active')
11];
12
13 constructor(private logger: LoggerService){
14
15 }
16
17 OnUserDetailsClicked: EventEmitter<User> = new EventEmitter<User>();
18
19 OnShowUserDetails(user: User){
20 this.OnUserDetailsClicked.emit(user);
21 }
22
23 GetAllUsers(){
24 return this.users;
25 }


File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
user-list.component.html
user.service.ts
user-detail.component.ts
app.module.ts
OPEN EDITORS
ANGULAR-SERVICES-DEPENDENCY...
user-detail.component...
user-list
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
app.module.ts
assets
favicon.ico
TIMELINE
src > app > header > admin > user-detail > user-detail.component.ts > UserDetailComponent > userService
1 import { Component, OnInit, inject } from '@angular/core';
2 import { User } from 'src/app/Models/User';
3 import { UserService } from 'src/app/Services/user.service';
4 import { USER_TOKEN } from 'src/app/app.module';
5
6 @Component({
7 selector: 'app-user-detail',
8 templateUrl: './user-detail.component.html'
9 })
10 export class UserDetailComponent implements OnInit {
11 selectedUser: User;
12 userService = inject(USER_TOKEN);
13
14 ngOnInit(){
15 this.userService.OnUserDetailsClicked.subscribe((data: User) => {
16 this.selectedUser = data;
17 console.log(this.selectedUser);
18 })
19 }
20 }
21


File Edit Selection View Go
angular-services-dependency-injection
EXPLORER
r-list.component.html
user.service.ts
user-detail.component.ts
user-detail.component.html
OPEN EDITORS 1 unsaved
ANGULAR-SERVICES-DEPENDENCY...
admin
user-detail
user-detail.component...
user-detail.component...
user-list
admin.component.html
admin.component.ts
home
header.component.html
header.component.ts
Models
Services
logger.service.ts
subscribe.service.ts
user.service.ts
app.component.css
app.component.html
app.component.spec.ts
app.component.ts
src > app > header > admin > user-detail > user-detail.component.html > div.user-detail-container
Go to component
<div class="user-detail-container" *ngIf="selectedUser">

<img class="user-detail-avatar-image" [src]="'./assets/images/'+selectedUser.gender.toL

<h2>User Detail</h2>
Name: {{ selectedUser.name }}
Gender: {{ selectedUser.gender }}
Subscription Type: {{ selectedUser.subType }}
Status: {{ selectedUser.status }}